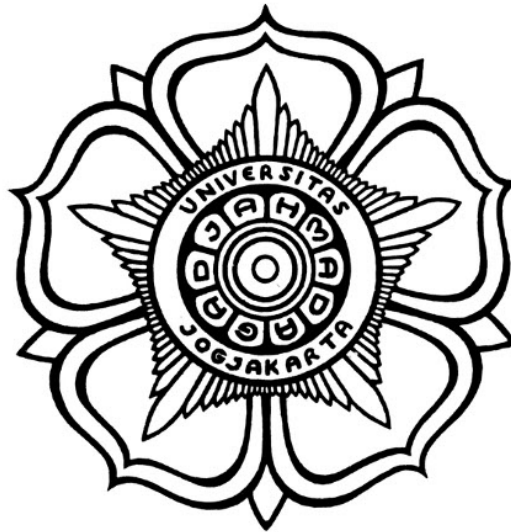


**PERANCANGAN PERIPHERAL I2S DENGAN AXI4-LITE
INTERFACE MENGGUNAKAN VERILOG**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:
Muhammad Muqtada Alhaddad

22/500341/TK/54841

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

PERANCANGAN PERIPHERAL I2S DENGAN AXI4-LITE INTERFACE MENGGUNAKAN VERILOG

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Muqtada Alhaddad
22/500341/TK/54841

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.

Ir. Wahyu Dewanto, M.T.

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhammad Muqtada Alhaddad
NIM : 22/500341/TK/54841
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 26 Mei 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Muqtada Alhaddad
22/500341/TK/54841

HALAMAN PERSEMBAHAN

Skripsi ini penulis persembahkan kepada:

Orang tua tercinta atas doa, dukungan, dan pengorbanan tanpa henti.

Para dosen pembimbing yang telah membimbing perjalanan akademik penulis.

Semua pihak yang telah berkontribusi dalam kesuksesan penelitian ini.

KATA PENGANTAR

"Dan tidaklah kamu diberi ilmu melainkan hanya sedikit." (QS. Al-Isra: 85)

"And of knowledge, you have been given only a little." (Quran 17:85)

Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini dengan judul **"Perancangan Peripheral I2S dengan AXI4-Lite Interface menggunakan Verilog"** telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Kedua orang tua dan keluarga besar yang senantiasa memberikan doa, dukungan moril, serta kasih sayang yang tak terhingga sepanjang masa studi penulis.
2. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM. dan Bapak Ir. Wahyu Dewanto, M.T. selaku dosen pembimbing yang telah memberikan bimbingan, arahan, serta waktu luangnya untuk membantu penulis menyelesaikan penelitian ini.
3. Segenap dosen dan staf di Departemen Teknik Elektro dan Teknologi Informasi (DTETI) FT UGM yang telah memberikan bekal ilmu pengetahuan dan bantuan administratif selama masa perkuliahan.
4. Teman-teman seperjuangan Teknologi Informasi angkatan 2022 yang telah memberikan semangat dan kerja sama selama masa studi.

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan, khususnya di bidang sistem digital dan sistem embedded, serta bagi kita semua, aamiin.

Yogyakarta, 14 Mei 2026

Muhammad Muqtada Alhaddad

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR SINGKATAN.....	xiii
INTISARI.....	xiv
ABSTRACT	xv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.3.1 Tujuan perancangan dan integrasi perangkat keras	3
1.3.2 Tujuan integrasi SoC dan perangkat lunak.....	3
1.3.3 Tujuan verifikasi dan validasi.....	3
1.4 Batasan Penelitian	3
1.4.1 Batasan perangkat keras dan RTL	4
1.4.2 Batasan SoC dan perangkat lunak.....	4
1.4.3 Batasan pengujian dan dokumentasi	4
1.5 Manfaat Penelitian	4
1.5.1 Manfaat akademik	4
1.5.2 Manfaat praktis	5
1.5.3 Manfaat pengembangan keahlian	5
1.6 Sistematika Penulisan.....	5
BAB II Tinjauan Pustaka dan Dasar Teori	6
2.1 Tinjauan Pustaka	6
2.1.1 Penelitian tentang implementasi I2S pada FPGA	6
2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC	6
2.1.3 Penelitian tentang sinkronisasi antar-domain clock (CDC)	7
2.1.4 Gap penelitian dan kontribusi	7
2.2 Dasar Teori	8
2.2.1 Protokol komunikasi I2S	8
2.2.1.1 Sinyal I2S	8
2.2.1.2 Format data Philips I2S.....	8

2.2.1.3	Relasi <i>sample rate</i> , BCLK, dan format slot	9
2.2.2	DAC audio PCM5102A.....	9
2.2.3	FPGA dan Basys3.....	10
2.2.4	MicroBlaze V dan memory-mapped I/O.....	10
2.2.5	Antarmuka AXI4-Lite	10
2.2.6	Sinkronisasi lintas domain clock (CDC).....	11
2.2.7	Clocking Wizard dan frekuensi aktual	12
2.2.8	Verilog HDL dan perancangan RTL	12
2.2.9	Analisis pemilihan metode	12
BAB III Metode Penelitian.....		14
3.1	Spesifikasi dan arsitektur IP I2S TX	14
3.1.1	Daftar fitur IP.....	14
3.1.2	Blok diagram perancangan IP	14
3.1.3	Register map	17
3.1.4	Aturan packing data.....	17
3.1.5	Integrasi Vivado Block Design	19
3.2	Alat dan Bahan.....	19
3.2.1	Perangkat keras.....	19
3.2.1.1	Board FPGA	19
3.2.1.2	Modul DAC audio	19
3.2.1.3	Perangkat bantu	19
3.2.2	Perangkat lunak	20
3.3	Metode yang Digunakan.....	20
3.3.1	Metode perancangan hardware	20
3.3.2	Metode verifikasi dan simulasi	20
3.3.3	Metode integrasi SoC	22
3.3.4	Metode pengembangan software	22
3.3.5	Metode pengujian hardware	22
3.3.6	Pengukuran dan evaluasi.....	23
3.4	Alur Tugas Akhir	23
BAB IV Hasil dan Pembahasan.....		25
4.1	Implementasi RTL dan Integrasi Vivado.....	25
4.1.1	Pemetaan pin (constraint).....	26
4.1.2	Detail logika RTL (serializer dan CDC)	27
4.1.2.1	Mekanisme Clock Domain Crossing (CDC)	28
4.1.2.2	Logika serializer Philips I2S	29
4.1.3	Diagram modul PCM5102A.....	30
4.2	Konfigurasi Clocking Wizard	30
4.3	Pengujian Sinyal dengan ILA	31

4.3.1	Verifikasi tambahan menggunakan testbench.....	32
4.4	Analisis Frekuensi Sampling	32
4.5	Identifikasi dan Perbaiki Bug Firmware	34
4.5.1	Bug 1: Masker Merusak Bit Tanda Sampel.....	34
4.5.1.1	Gejala.....	34
4.5.1.2	Penyebab	35
4.5.1.3	Solusi	35
4.5.2	Bug 2: Mode 32-bit Kehilangan 1 Bit Resolusi	35
4.5.2.1	Gejala.....	35
4.5.2.2	Penyebab	35
4.5.2.3	Solusi	35
4.5.3	Bug 3: Kanal Kanan Mute saat Mode Right-Only	36
4.5.3.1	Gejala.....	36
4.5.3.2	Penyebab	36
4.5.3.3	Solusi	36
4.5.4	Bug 4: Frekuensi Audio Lebih Rendah dari Target	36
4.5.4.1	Gejala.....	36
4.5.4.2	Penyebab	36
4.5.4.3	Solusi	37
4.6	Verifikasi Setelah Perbaikan	38
4.7	Pembahasan terhadap Tujuan Penelitian	38
BAB V	Tambahan (Opsional)	41
5.1	Perbandingan dengan referensi desain	41
BAB VI	Kesimpulan dan Saran	42
6.1	Kesimpulan	42
6.2	Saran	42
DAFTAR PUSTAKA		44
LAMPIRAN		L-1
L.1	Hasil resource utilization	L-1
L.1.1	Post-implementation resource report.....	L-1
L.1.2	Timing report.....	L-1
L.2	Timing diagram	L-1
L.2.1	I2S audio transmission timing	L-1
L.3	Log hasil pengujian	L-2
L.3.1	I2S audio playback test	L-2
L.4	Deskripsi block design.....	L-2
L.4.1	Diagram Block (Color-coded)	L-3
L.4.2	I2S IP Block (Standalone)	L-3
L.4.3	Clocking View.....	L-4

L.4.4	Interface View	L-4
L.5	Hasil pengujian i2s transmitter - capture osiloskop	L-5
L.5.1	Test Frekuensi Audio	L-5
L.5.2	Test Kontrol Enable	L-6
L.5.3	Test Kontrol Mute	L-7
L.5.4	Test Routing Stereo	L-8
L.5.5	Test Lebar Sample (Sample Width)	L-9
L.6	Design Timing Summary	L-10
L.7	Power Summary	L-10
Block Design IP Summaries		L-12
BAB L	Kode Sumber Perangkat Lunak (Vitis)	L-14
BAB L	Kode Sumber Perangkat Keras (Verilog)	L-18
L.1	Top-level Wrapper (i2s.v)	L-18
L.2	Modul AXI4-Lite Slave dan Serializer (i2s_slave_lite_v1_0_S00_AXI.v)	L-19

DAFTAR TABEL

Tabel 3.1	Register map IP I2S AXI4-Lite.	17
Tabel 3.2	Bitfield register <code>CONTROL</code>	17
Tabel 4.1	Konfigurasi clock hasil Clocking Wizard.....	31
Tabel 4.2	Hasil pengujian clock I2S pada keluarga frekuensi 48 kHz.	33
Tabel 4.3	Perbandingan akurasi timing loop sebelum dan sesudah perbaikan...	38
Tabel 4.4	Hasil verifikasi firmware setelah perbaikan bug.....	39
Tabel L.1	Ringkasan utilisasi resource FPGA.....	L-1
Tabel L.2	Hasil analisis timing	L-1

DAFTAR GAMBAR

Gambar 1.1	Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.	2
Gambar 3.2	Blok diagram internal IP I2S TX.	14
Gambar 3.3	Bit diagram register CONTROL (bit 31..13 RESERVED, 12..8 SAMPLE_WIDTH, 7..4 RESERVED, 3 FS_MODE, 2 FS_FAMILY, 1 MUTE, 0 ENABLE).	17
Gambar 3.4	Integrasi IP I2S pada Block Design.	19
Gambar 3.5	Diagram alir tahapan penelitian tugas akhir.	24
Gambar 4.6	Block Design Vivado: MicroBlaze V terhubung ke IP I2S melalui AXI SmartConnect.	25
Gambar 4.7	Wiring PMOD JA ke modul PCM5102A.	27
Gambar 4.8	Diagram blok fungsional PCM5102A dari dokumentasi Texas Instruments.	31
Gambar 4.9	Tangkapan ILA: BCLK, WS, dan DATA – satu frame stereo 64 BCLK.	32
Gambar 4.10	Keluaran audio setelah perbaikan: kanal L dan R memiliki amplitudo dan frekuensi identik.	38
Gambar 4.11	Mode Right-only setelah perbaikan: kanal kanan aktif, kanal kiri = 0.	39
Gambar 4.12	Verifikasi frekuensi: osiloskop/ILA tangkapan pada 1 kHz, 2 kHz, dan 5 kHz setelah perbaikan.	39
Gambar 5.13	Ringkasan timing desain.	41
Gambar L.1	Diagram block warna-warni arsitektur SoC.	L-3
Gambar L.2	Blok IP I2S standalone.	L-4
Gambar L.3	Clocking view dan jalur pemilihan clock audio.	L-4
Gambar L.4	Interface view pemetaan pin dan jalur debug.	L-5
Gambar L.5	Capture osiloskop: audio tone 1 kHz.	L-5
Gambar L.6	Capture osiloskop: audio tone 2 kHz.	L-6
Gambar L.7	Capture osiloskop: audio tone 5 kHz.	L-6
Gambar L.8	Capture osiloskop: enable=0.	L-6
Gambar L.9	Capture osiloskop: enable=1.	L-7
Gambar L.10	Capture osiloskop: mute=0.	L-7
Gambar L.11	Capture osiloskop: mute=1.	L-7
Gambar L.12	Hasil capture osiloskop: Left channel only - hanya kanal left dioutput.	L-8
Gambar L.13	Hasil capture osiloskop: Right channel only - hanya kanal right dioutput.	L-8
Gambar L.14	Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan.	L-8
Gambar L.15	Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit.	L-9
Gambar L.16	Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit.	L-9
Gambar L.17	Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame.	L-10

Gambar L.18	Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure	L-10
Gambar L.19	Power summary report - total on-chip power consumption, bre- akdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins	L-11

DAFTAR SINGKATAN

AXI	=	Advanced eXtensible Interface
BCLK	=	Bit Clock
BRAM	=	Block RAM
CDC	=	Clock Domain Crossing
CPU	=	Central Processing Unit
DAC	=	Digital to Analog Converter
DMA	=	Direct Memory Access
DSP	=	Digital Signal Processor / Processing
FF	=	Flip-Flop
FIFO	=	First-In-First-Out
FPGA	=	Field Programmable Gate Array
HDL	=	Hardware Description Language
I2S	=	Inter-IC Sound
ILA	=	Integrated Logic Analyzer
IP	=	Intellectual Property
IRQ	=	Interrupt Request
LRCK	=	Left/Right Clock (sama dengan WS)
LSB	=	Least Significant Bit
LUT	=	Look-Up Table
MCLK	=	Master Clock
MMCM	=	Mixed-Mode Clock Manager
MSB	=	Most Significant Bit
RISC-V	=	Reduced Instruction Set Computer V
RTL	=	Register Transfer Level
SoC	=	System on Chip
UART	=	Universal Asynchronous Receiver-Transmitter
WS	=	Word Select
XDC	=	Xilinx Design Constraints

INTISARI

Penelitian ini berfokus pada perancangan dan implementasi peripheral transmitter I2S (Inter-IC Sound) menggunakan bahasa Verilog HDL yang diintegrasikan ke dalam System-on-Chip (SoC) berbasis prosesor MicroBlaze V pada FPGA. Peripheral ini diimplementasikan sebagai custom IP dengan antarmuka AXI4-Lite slave sehingga dapat dikonfigurasi dan diisi data audio melalui register memory-mapped oleh perangkat lunak yang berjalan di Xilinx Vitis. Platform target yang digunakan adalah board FPGA Basys3, sedangkan keluaran audio direalisasikan melalui modul DAC PCM5102A. Modul pendukung seperti UART dan infrastruktur clock hanya digunakan sebagai penunjang sistem, sementara kontribusi utama skripsi ini adalah desain RTL, integrasi, dan validasi peripheral transmitter I2S.

Metodologi penelitian mencakup: (1) desain RTL Verilog untuk transmitter I2S yang mengikuti format Philips I2S untuk transmisi audio stereo, (2) perancangan antarmuka register AXI4-Lite untuk kontrol, pemuatan sampel, dan observasi status, (3) implementasi buffer berbasis FIFO dan interupsi watermark untuk mengurangi risiko underrun, (4) integrasi custom IP I2S ke dalam SoC MicroBlaze V menggunakan Vivado block design, (5) pengembangan perangkat lunak uji dalam bahasa C pada Xilinx Vitis untuk konfigurasi register, prefill FIFO, dan pengiriman sampel audio, serta (6) verifikasi melalui synthesis, implementation, pengamatan Integrated Logic Analyzer (ILA), dan pengujian hardware dengan modul PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S yang dirancang dapat diintegrasikan dengan baik ke dalam SoC berbasis MicroBlaze V dan dioperasikan melalui kontrol memory-mapped AXI4-Lite. Desain yang diimplementasikan mampu membangkitkan sinyal I2S yang diperlukan, mentransmisikan sampel audio stereo dalam format Philips I2S, serta mendukung dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz yang diturunkan dari konfigurasi audio clock. Validasi perangkat keras menunjukkan bahwa desain bekerja sesuai tujuan pada platform yang diuji dan dapat menjadi dasar yang dapat digunakan kembali untuk sistem keluaran audio digital berbasis FPGA.

Kata kunci : MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Audio Digital

ABSTRACT

This research focuses on the design and implementation of an I2S (Inter-IC Sound) transmitter peripheral using Verilog HDL, integrated into a System-on-Chip (SoC) based on the MicroBlaze V processor on FPGA. The peripheral is implemented as a custom AXI4-Lite slave IP so that it can be configured and supplied with audio data through memory-mapped registers from software running in Xilinx Vitis. The target platform is the Basys3 FPGA board, while audio output is generated through a PCM5102A DAC module. Supporting modules such as UART and clocking infrastructure are included only as system support, whereas the main contribution of the thesis is the RTL design, integration, and validation of the I2S transmitter peripheral.

The methodology includes: (1) Verilog RTL design of an I2S transmitter that follows the Philips I2S format for stereo audio transmission, (2) design of an AXI4-Lite register interface for control, sample loading, and status observation, (3) implementation of FIFO-based buffering and watermark interrupt support to reduce underrun risk, (4) integration of the custom I2S IP into a MicroBlaze V SoC using Vivado block design, (5) development of C test software in Xilinx Vitis for register configuration, FIFO prefill, and audio sample transmission, and (6) verification through synthesis, implementation, Integrated Logic Analyzer (ILA) observation, and hardware testing with the PCM5102A module.

The results show that the designed I2S transmitter IP can be integrated successfully into a MicroBlaze V-based SoC and operated through AXI4-Lite memory-mapped control. The implemented design is able to generate the required I2S signals, transmit stereo audio samples in Philips I2S format, and support two nominal sampling-rate modes around 48 kHz and 96 kHz derived from the audio clock configuration. Hardware validation indicates that the design functions as intended on the tested platform and provides a reusable basis for FPGA-based digital audio output systems.

Keywords: MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Digital Audio

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

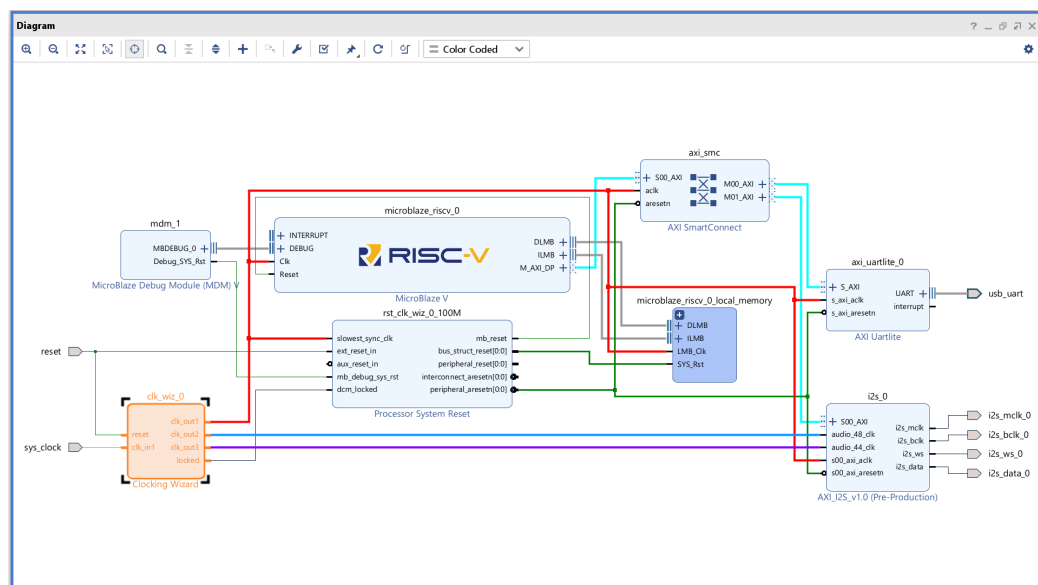
I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain [1]. Protokol ini menggunakan tiga sinyal utama, yaitu *bit clock* (BCLK), *word select* (WS/LRCLK), dan *serial data* (SD). Relasi timing antar sinyal tersebut harus dipenuhi dengan ketat agar penerima audio dapat merekonstruksi sampel digital dengan benar. Dukungan terhadap berbagai *sample rate* dan *bit depth* membuat I2S cocok untuk implementasi audio pada FPGA, terutama ketika dibutuhkan fleksibilitas desain pada level register transfer.

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi [2]. Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA memungkinkan perancangan antarmuka komunikasi, logika buffer, pembagi clock, dan jalur data serial secara langsung dalam Verilog atau VHDL [3]. Ketika dikombinasikan dengan soft processor seperti MicroBlaze V, FPGA juga memberikan model kerja hardware-software co-design: fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara konfigurasi, pengiriman data, dan pengujian dikendalikan oleh perangkat lunak [4].

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V [5] pada ekosistem AMD/Xilinx [6] menyediakan fondasi yang sesuai untuk integrasi peripheral kustom pada Vivado dan Vitis. Melalui antarmuka AXI4-Lite [7], peripheral dapat diakses dengan model memory-mapped I/O yang sederhana namun efektif. Dengan pendekatan ini, prosesor dapat menulis sampel audio, mengatur mode operasi, serta membaca status dari peripheral I2S menggunakan instruksi load/store biasa. Pendekatan tersebut memudahkan pengembangan perangkat lunak uji sekaligus menjaga modularitas desain perangkat keras.

Dalam praktik implementasi audio pada FPGA, permasalahan tidak hanya terletak pada pembangkitan sinyal I2S, tetapi juga pada kestabilan aliran data. Jika sampel audio tidak tersedia tepat saat serializer membutuhkan data baru, maka akan terjadi *underrun* yang menghasilkan gangguan audio. Oleh karena itu, sistem membutuhkan strategi buffering yang memadai, misalnya FIFO TX dan interupsi *watermark*, sehingga prosesor dapat mengisi ulang data sebelum buffer kosong. Selain itu, akurasi frekuensi clock hasil Clocking Wizard [8] dan kesesuaian pinout ke modul DAC eksternal [9] juga menjadi faktor penting yang memengaruhi keberhasilan sistem secara keseluruhan.

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan **IP core transmitter I2S berbasis Verilog** dengan antarmuka **AXI4-Lite** yang dapat diintegrasikan ke dalam SoC **MicroBlaze V** pada board **Basys3** [10]. Kode sumber dan dokumentasi proyek ini tersedia secara terbuka pada repositori GitHub¹. Target keluaran audio adalah modul **PCM5102A** sebagai DAC eksternal. Desain difokuskan pada mode **TX-only**, format **Philips I2S**, stereo, dengan **24 bit per saluran** dalam **slot 32 bit**, serta dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz. Penelitian ini mencakup perancangan RTL, integrasi Vivado Block Design [11], pengembangan perangkat lunak uji di Vitis [12], serta validasi melalui ILA [13], pengukuran sinyal, dan bukti keluaran audio.



Gambar 1.1. Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

¹<https://github.com/adakhaddad/thes2is>

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka **AXI4-Lite** sehingga dapat dikontrol dari program Vitis melalui register memory-mapped?
2. Bagaimana mendukung **dua mode frekuensi sampling** (sekitar 48 kHz dan 96 kHz) dari satu `audio_clk` dengan pembagi yang dapat dipilih, sambil memperhatikan frekuensi aktual hasil Clocking Wizard?
3. Bagaimana merancang **FIFO TX** dan **interupsi watermark** untuk meminimalkan *underrun* dan mengurangi beban CPU saat pengisian data audio?
4. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, **ILA**, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan.

1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan modul **I2S TX** dengan **AXI4-Lite slave**, register kontrol dan data, serializer **Philips I2S**, serta **FIFO TX** untuk mendukung pengiriman sampel audio stereo.
2. Mengimplementasikan **pembagi clock** untuk dua mode frekuensi sampling nominal dan mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S melalui **Vitis**.
2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, **prefill FIFO**, penanganan **ISR** isi ulang FIFO, dan pemutaran sinyal uji audio.

1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui **ILA**, pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, level FIFO, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dibatasi pada **dua mode** nominal, yaitu sekitar 48 kHz dan 96 kHz, yang berasal dari satu sumber `audio_clk`.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register/FIFO dan tidak mencakup integrasi **DMA**.
5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada ILA, osiloskop bila tersedia, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai data-sheet komersial lengkap.

1.5 Manfaat Penelitian

1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi buffering, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **FIFO + watermark interrupt** yang dapat diadaptasi pada peripheral streaming lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.
2. Mengembangkan kemampuan debugging hardware-software melalui ILA, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

1.6 Sistematika Penulisan

Bab I membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

Bab II memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, buffering FIFO, Clocking Wizard, MicroBlaze V, dan PCM5102A.

Bab III menyajikan metode penelitian dan rancangan sistem, termasuk alat dan bahan, spesifikasi IP, register map, dan integrasi pada Vivado.

Bab IV berisi implementasi RTL, integrasi sistem, hasil pengujian ILA, pengukuran, dan validasi audio.

Bab V memuat pembahasan tambahan atau analisis opsional jika diperlukan.

Bab VI berisi kesimpulan dan saran pengembangan.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan AXI4-Lite sebagai antarmuka register.

2.1.1 Penelitian tentang implementasi I2S pada FPGA

Implementasi antarmuka I2S pada FPGA telah dilaporkan dalam beberapa konteks. [14] merancang antarmuka I2S digital audio pada FPGA untuk menghubungkan prosesor dengan codec audio, menunjukkan bahwa protokol I2S dapat diimplementasikan dengan latensi rendah dan mendukung *sample rate* hingga 96 kHz. Penelitian tersebut menggunakan pendekatan RTL langsung tanpa antarmuka AXI, sehingga berbeda dari pendekatan berbasis register yang digunakan pada penelitian ini.

[15] mengintegrasikan blok logika I2S bersama *embedded logic analyzer* pada FPGA berbiaya rendah. Kontribusi penelitian tersebut terutama pada sisi instrumentasi, sedangkan fokus penelitian ini adalah pada desain peripheral yang dapat dikontrol dari perangkat lunak melalui antarmuka AXI4-Lite.

Secara umum, penelitian-penelitian tersebut mengkonfirmasi kelayakan implementasi I2S pada FPGA dan menjadi landasan pemilihan arsitektur serializer pada penelitian ini. Namun demikian, sebagian besar publikasi tersebut berhenti pada pembuktian bahwa sinyal I2S dapat dibangkitkan secara benar pada level RTL atau pada pengujian modul terisolasi. Penelitian ini mengambil lingkup yang lebih sistemik, yakni mengkaji bagaimana blok I2S yang sama dapat dikemas sebagai peripheral yang dapat diprogram, diintegrasikan ke lingkungan SoC, dan divalidasi melalui perangkat lunak nyata yang berjalan pada prosesor soft-core. Dengan demikian, perbedaan utama penelitian ini bukan hanya pada implementasi protokol, melainkan pada pendekatan *hardware-software co-design* yang menyatukan register map, integrasi bus, dan validasi end-to-end. Perspektif sistem semacam ini juga sejalan dengan pendekatan desain *embedded FPGA* yang menempatkan integrasi IP, perangkat lunak, dan alur toolchain sebagai satu kesatuan proses pengembangan [16].

2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

Waterman dkk. [5] memperkenalkan arsitektur RISC-V sebagai ISA terbuka yang modular dan ekstensibel, yang kemudian mendasari pengembangan banyak prosesor soft-core termasuk MicroBlaze V pada ekosistem AMD/Xilinx [6]. Ketersediaan antarmuka

AXI4-Lite yang terintegrasi pada MicroBlaze V menjadi dasar pemilihan prosesor ini dalam penelitian ini.

Sklyarov dkk. [17] mengeksplorasi perancangan sistem SoC hierarkis berbasis bus AXI pada FPGA. Penelitian tersebut menunjukkan bahwa peripheral kustom dapat diintegrasikan ke dalam SoC melalui AXI interconnect dengan cara yang modular dan dapat diskalakan – pendekatan yang sama diterapkan dalam penelitian ini menggunakan Vivado IP Integrator.

Crockett dkk. [4] memberikan gambaran komprehensif mengenai hardware-software co-design pada FPGA berbasis prosesor, termasuk pola pengembangan peripheral AXI dan aplikasi bare-metal melalui SDK. Meskipun konteks platform berbeda (Zynq vs. Basys3), metodologi integrasi yang diulas relevan dengan pendekatan yang digunakan pada penelitian ini.

2.1.3 Penelitian tentang sinkronisasi antar-domain clock (CDC)

Dalam sistem digital yang melibatkan lebih dari satu domain clock – seperti IP I2S yang memiliki domain AXI (100 MHz) dan domain audio (~ 24 MHz) – sinkronisasi lintas domain clock (*Clock Domain Crossing*, CDC) menjadi aspek kritikal.

Ginosar [18] memberikan analisis mendalam tentang masalah metastabilitas pada sistem digital dan teknik sinkronisasi flip-flop ganda yang umum digunakan. Prinsip ini diterapkan langsung pada RTL penelitian ini melalui mekanisme sinkronisasi write-counter 4-bit dari domain AXI ke domain audio.

2.1.4 Gap penelitian dan kontribusi

Berdasarkan tinjauan di atas, dapat diidentifikasi beberapa celah penelitian:

1. Dokumentasi lengkap mengenai pengembangan **IP I2S TX custom** untuk **MicroBlaze V** yang mencakup RTL, register map, integrasi Vivado, aplikasi Vitis, validasi hardware, dan identifikasi bug firmware masih relatif terbatas.
2. Sebagian besar penelitian yang ada menggunakan platform prosesor yang lebih besar (Zynq, Cortex-A) sehingga contoh pada platform sederhana berbasis Basys3 memberikan nilai referensi tersendiri.
3. Analisis bug pada lapisan firmware – khususnya justifikasi bit sampel dan timing loop berbasis cycle counter – belum banyak didokumentasikan secara eksplisit dalam konteks I2S pada FPGA.

Jika diringkas, penelitian terdahulu telah menunjukkan tiga hal penting: I2S dapat diimplementasikan secara andal pada FPGA, peripheral kustom dapat dihubungkan ke SoC berbasis AXI, dan CDC merupakan aspek kritis pada sistem multi-clock. Akan tetapi,

dokumentasi yang menyatukan ketiganya pada satu alur yang lengkap masih terbatas. Keterbatasan inilah yang menjadi dasar penyusunan penelitian ini, sehingga hasil yang diharapkan bukan hanya berupa IP yang berfungsi, tetapi juga uraian langkah integrasi dan validasi yang dapat direplikasi.

Kontribusi penelitian ini adalah:

1. Merancang dan mengimplementasikan **IP core I2S transmitter** berbasis Verilog dengan antarmuka **AXI4-Lite** yang kompatibel dengan **MicroBlaze V**.
2. Menyediakan alur lengkap dari desain RTL, integrasi SoC, pengembangan perangkat lunak uji, hingga validasi pada hardware Basys3 dengan PCM5102A.
3. Mengidentifikasi dan mendokumentasikan tiga bug firmware (justifikasi bit, masking tanda, timing loop) beserta solusinya yang dapat menjadi referensi praktis.
4. Menyajikan analisis akurasi clock audio berbasis Clocking Wizard Artix-7 yang dapat digunakan kembali pada desain serupa.

2.2 Dasar Teori

2.2.1 Protokol komunikasi I2S

I2S (*Inter-IC Sound*) adalah protokol serial sinkron yang dirancang oleh Philips untuk mengirimkan data audio digital antarperangkat [1]. I2S digunakan secara luas untuk menghubungkan prosesor, FPGA, codec, DAC, dan ADC audio.

2.2.1.1 Sinyal I2S

I2S menggunakan tiga sinyal utama:

- **Serial Data (SD/DATA)**: membawa data audio secara serial, MSB first.
- **Word Select (WS/LRCLK)**: menandai kanal kiri (LOW) dan kanan (HIGH); frekuensinya sama dengan *sample rate*.
- **Bit Clock (BCLK/SCK)**: clock serial yang menentukan perpindahan bit data.

Beberapa implementasi juga menggunakan sinyal **MCLK** (*Master Clock*) sebagai referensi frekuensi bagi perangkat penerima, meskipun banyak DAC modern seperti PCM5102A dapat beroperasi tanpa MCLK menggunakan mode SCK [9].

2.2.1.2 Format data Philips I2S

Pada format Philips I2S [1]:

- data dikirim **MSB first**;
- transisi WS terjadi **satu periode BCLK sebelum MSB** kanal berikutnya (*one-bit delay*);

- bit pertama setelah peralihan WS selalu bernilai nol (bit tunda wajib);
- data kiri dan kanan dikirim bergantian secara serial;
- implementasi praktis menggunakan slot 16, 24, atau 32 bit per kanal.

Pada penelitian ini, data audio dikirim sebagai **stereo 24 bit per saluran dalam slot 32 bit** sehingga satu frame stereo memerlukan 64 pulsa BCLK, dan lebar slot dapat dikonfigurasi melalui field `SAMPLE_WIDTH` pada register `CONTROL`. Pemilihan format 24-bit dalam slot 32-bit memberikan dua keuntungan praktis. Pertama, format ini umum dipakai oleh DAC audio modern sehingga kompatibilitas perangkat lebih mudah dipenuhi. Kedua, slot 32-bit menyederhanakan logika serializer dan pemetaan register 32-bit pada AXI4-Lite, karena satu transaksi tulis dapat langsung memuat satu sampel kanal tanpa perlu pemadatan bit tambahan di sisi perangkat lunak.

2.2.1.3 Relasi *sample rate*, BCLK, dan format slot

Frekuensi sampling F_s menentukan jumlah frame audio stereo per detik. Jika satu frame menggunakan N bit clock:

$$F_s = \frac{f_{\text{BCLK}}}{N} \quad (2-1)$$

Untuk format stereo 32-bit per kanal, $N = 64$. Hubungan antara clock master dan BCLK dinyatakan sebagai:

$$f_{\text{BCLK}} = \frac{f_{\text{MCLK}}}{\text{divider}} \quad (2-2)$$

Dengan $f_{\text{MCLK}} = 24,576$ MHz dan divider 8, diperoleh $f_{\text{BCLK}} = 3,072$ MHz dan $F_s = 48$ kHz.

2.2.2 DAC audio PCM5102A

PCM5102A adalah DAC audio stereo yang menerima data digital melalui antarmuka audio serial I2S [9]. Spesifikasi utama yang relevan:

- Mendukung *sample rate* hingga 384 kHz dan resolusi hingga 32 bit.
- Dapat beroperasi tanpa MCLK eksternal (mode SCK).
- Kompatibel dengan tegangan logika 3,3 V, cocok untuk antarmuka PMOD Basys3.
- Memiliki output analog stereo langsung tanpa filter eksternal tambahan.

Dalam konteks penelitian ini, PCM5102A berperan sebagai perangkat penerima data I2S dari FPGA. Keberhasilan sistem bergantung pada ketepatan timing BCLK, WS, dan DATA.

2.2.3 FPGA dan Basys3

FPGA (*Field-Programmable Gate Array*) adalah perangkat semikonduktor yang logika internalnya dapat dikonfigurasi ulang setelah fabrikasi [2]. Keunggulan FPGA untuk sistem audio digital mencakup kemampuan mengimplementasikan logika serial waktu nyata dengan latensi deterministik, pembangkitan clock presisi melalui PLL/MMCM internal, serta integrasi antara perangkat keras dan perangkat lunak melalui prosesor soft-core [19].

Basys3 adalah board FPGA berbasis Xilinx Artix-7 (XC7A35T) yang digunakan sebagai platform implementasi utama pada penelitian ini [10]. Board ini menyediakan osilator 100 MHz, antarmuka USB-UART, dan header PMOD untuk koneksi ke perangkat eksternal.

2.2.4 MicroBlaze V dan memory-mapped I/O

MicroBlaze V adalah prosesor soft-core berbasis RISC-V yang terintegrasi dalam ekosistem Vivado dan Vitis [6]. Peripheral kustom diakses melalui model **memory-mapped I/O**: register peripheral dipetakan ke alamat tertentu dalam ruang alamat prosesor, sehingga dapat dibaca dan ditulis menggunakan instruksi `load/store` biasa.

Dengan pendekatan ini, perangkat lunak dapat:

- menulis register `CONTROL` untuk mengaktifkan peripheral dan mengatur mode;
- mengisi register `DATA_LEFT/DATA_RIGHT` dengan sampel audio;
- membaca register untuk verifikasi *readback*.

Pada penelitian ini, keberadaan prosesor soft-core tidak hanya berfungsi sebagai penghasil sampel audio, tetapi juga sebagai sarana untuk mengevaluasi apakah peripheral benar-benar mudah dikendalikan dari perangkat lunak. Dengan kata lain, keberhasilan desain tidak cukup diukur dari sinyal I2S yang benar di simulator, tetapi juga dari kemampuan sistem untuk menerima konfigurasi, memproses perubahan mode, dan mempertahankan keluaran audio yang stabil saat dikendalikan melalui *memory-mapped I/O*.

RISC-V menyediakan *Control and Status Register* (CSR) `mcycle` yang merupakan counter siklus CPU 64-bit bebas-jalan (*free-running*) [20]. Register ini diakses melalui instruksi `rdcycle/rdcycleh` dan digunakan pada penelitian ini untuk implementasi timing loop dengan presisi 10 ns (pada clock 100 MHz).

2.2.5 Antarmuka AXI4-Lite

AXI4-Lite adalah subset sederhana dari protokol AMBA AXI4 yang ditujukan untuk peripheral berbasis register [7]. Karakteristik utama:

- Lima kanal independen: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), dan Read Data (R).
- Protokol handshake `VALID/READY` pada setiap kanal memungkinkan *backpressure* tanpa kehilangan data.
- Tidak mendukung *burst transfer* – setiap transaksi membaca atau menulis satu word 32-bit.
- Cocok untuk konfigurasi register peripheral dengan kebutuhan bandwidth rendah.

Dokumentasi implementasi AXI4-Lite pada ekosistem Xilinx tersedia pada [21]. AXI4-Lite dipilih dalam penelitian ini karena karakter peripheral yang dirancang lebih dekat ke pola konfigurasi register dibanding *streaming* berkecepatan tinggi. Register `CONTROL` digunakan untuk mengubah mode kerja, sedangkan `DATA_LEFT` dan `DATA_RIGHT` digunakan untuk memuat sampel saat validasi. Pendekatan ini menyederhanakan integrasi pada Vivado IP Integrator dan memudahkan penelusuran bug, meskipun konsekuensinya adalah throughput sistem tetap bergantung pada kecepatan CPU saat menulis sampel secara *programmed I/O*.

2.2.6 Sinkronisasi lintas domain clock (CDC)

Sistem I2S melibatkan dua domain clock yang tidak sinkron: domain AXI (100 MHz) dan domain audio (≈ 24 MHz). Data yang melintas antara dua domain dapat mengalami metastabilitas jika tidak disinkronkan dengan benar [18].

Teknik umum yang digunakan adalah **sinkronisasi flip-flop ganda**: sinyal dari domain sumber disampling dua kali berurutan oleh clock domain tujuan sebelum digunakan. Hal ini memberikan waktu yang cukup bagi flip-flop untuk keluar dari kondisi metastabil.

Pada penelitian ini, RTL menggunakan **write-counter 4-bit** yang bertambah pada setiap transaksi AXI write. Domain audio menyinkronkan counter ini dengan dua flip-flop dan mendeteksi setiap perubahan, sehingga setiap penulisan register dari CPU dijamin terdeteksi tanpa terlewat. Pemilihan clock audio melalui `BUFGMUX` juga disinkronkan terhadap domain AXI sebelum digunakan sebagai sinyal selektor. Pendekatan ini dipilih karena yang perlu dipindahkan antar-domain pada penelitian ini bukan aliran data lebar berkecepatan tinggi, melainkan peristiwa pembaruan register yang relatif jarang dan terstruktur. Oleh sebab itu, kombinasi write-counter, sinkronisasi dua flip-flop, dan *shadow register* pada batas frame sudah memadai untuk menjaga koherensi data tanpa kompleksitas FIFO asinkron penuh.

2.2.7 Clocking Wizard dan frekuensi aktual

Pada FPGA Xilinx, *Clocking Wizard* memanfaatkan MMCM (*Mixed-Mode Clock Manager*) untuk menghasilkan clock internal yang mendekati frekuensi target [8]. Nilai yang dihasilkan (*actual frequency*) umumnya sedikit berbeda dari nilai nominal karena keterbatasan rasio frekuensi MMCM. Selisih ini berpengaruh langsung pada F_s aktual dan perlu dicatat saat evaluasi hasil.

2.2.8 Verilog HDL dan perancangan RTL

Verilog digunakan untuk mendeskripsikan logika perangkat keras pada level RTL [3]. Pada penelitian ini, Verilog digunakan untuk membangun:

- antarmuka slave AXI4-Lite dengan mesin state tulis dan baca;
- mekanisme CDC berbasis write-counter;
- logika pemilih clock (BUFGMUX);
- serializer I2S sesuai spesifikasi Philips;
- logika *power-on reset* domain audio.

Prinsip desain digital yang digunakan merujuk pada [22] dan [23].

2.2.9 Analisis pemilihan metode

Metode yang dipilih pada penelitian ini didasarkan pada beberapa pertimbangan:

1. **MicroBlaze V** dipilih karena terintegrasi dengan baik pada ekosistem Vivado dan Vitis serta sesuai untuk sistem embedded bare-metal [6].
2. **AXI4-Lite** dipilih karena sesuai untuk peripheral berbasis register dengan konfigurasi sederhana dan didukung penuh oleh IP Integrator [21].
3. **Verilog HDL** dipilih karena umum digunakan untuk desain RTL pada FPGA dan memiliki dukungan sintesis yang baik pada Vivado [3].
4. Pendekatan **hardware-software co-design** dipilih agar peripheral dapat divalidasi tidak hanya dari sisi RTL, tetapi juga dari sisi penggunaan nyata oleh perangkat lunak [4].

Selain pertimbangan di atas, lingkup penelitian sengaja dibatasi pada jalur **TX-only** tanpa FIFO dan tanpa DMA. Pembatasan ini dilakukan agar fokus penelitian tetap berada pada ketepatan protokol I2S, keamanan CDC, dan integrasi peripheral kustom ke SoC berbasis MicroBlaze V. Konsekuensinya, sistem yang dihasilkan belum ditujukan untuk playback audio berdurasi panjang dengan throughput tinggi, melainkan sebagai platform validasi yang menekankan kejelasan arsitektur, kemudahan pengendalian, dan keterlancaran bug. Pertimbangan ini konsisten dengan literatur desain sistem FPGA embedded

yang menekankan pentingnya memilih arsitektur yang cukup sederhana untuk divalidasi menyeluruh sebelum diperluas ke fitur throughput yang lebih tinggi [16].

BAB III

METODE PENELITIAN

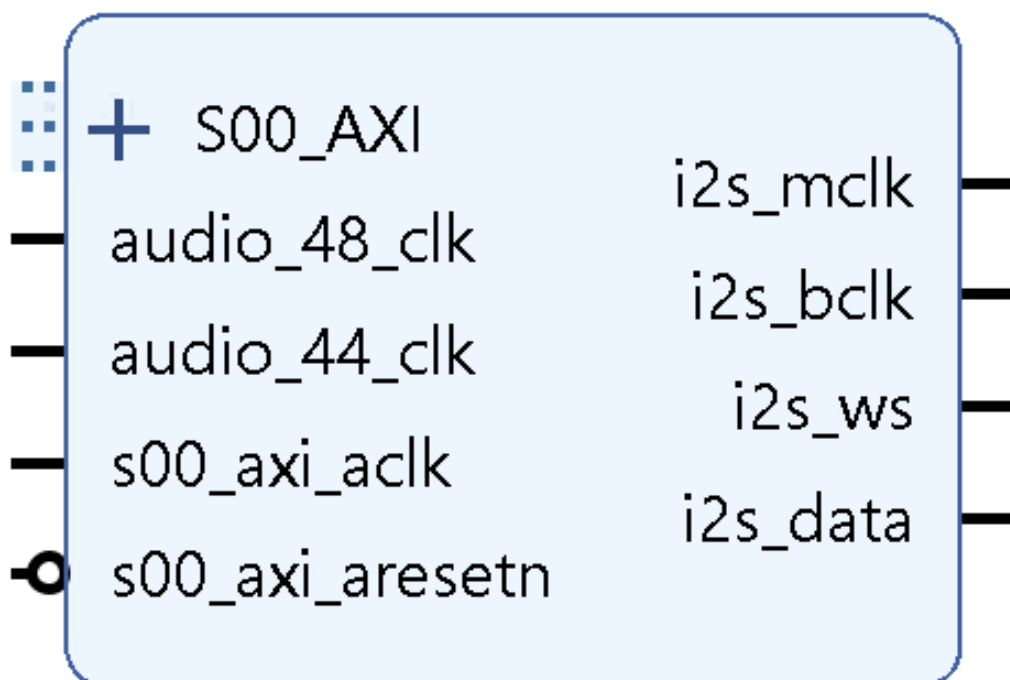
3.1 Spesifikasi dan arsitektur IP I2S TX

Bagian ini merangkum **perancangan fungsional** IP yang menjadi inti skripsi. Implementasi difokuskan pada peripheral **I2S transmitter** dengan antarmuka **AXI4-Lite** agar dapat dikontrol oleh program C pada MicroBlaze V [6].

3.1.1 Daftar fitur IP

- Antarmuka **AXI4-Lite** 32 bit untuk konfigurasi dan pengisian data audio [7].
- Mode **TX-only**, stereo, format **Philips I2S** [1], **24 bit** per saluran dalam **slot 32 bit**.
- Dukungan dua keluarga frekuensi sampling nominal (**48/96 kHz** dan **44,1/88,2 kHz**), melalui pembagi clock internal dan BUFGMUX [8].
- **Jalur data** lewat register `DATA_LEFT/DATA_RIGHT` dan `CONTROL`, **tanpa FIFO** internal.
- Keluaran menuju modul **PCM5102A** [9]: `BCLK`, `LRCK/WS`, `DATA`, dan `MCLK` bila digunakan.

3.1.2 Blok diagram perancangan IP



Gambar 3.2. Blok diagram internal IP I2S TX.

Alur kerja IP dimulai ketika perangkat lunak pada MicroBlaze V menulis register `DATA_LEFT`, `DATA_RIGHT`, dan `CONTROL` melalui transaksi AXI4-Lite. Nilai-nilai tersebut mula-mula disimpan pada domain `S_AXI_ACLK`, kemudian informasi pembaruan register dilewatkan ke domain audio melalui mekanisme CDC berbasis write-counter. Pada domain audio, sampel yang telah tersinkron diambil ke *shadow register* hanya saat batas frame stereo tercapai, sehingga data kanal kiri dan kanan selalu berubah secara serempak. Tahap terakhir adalah serializer yang mengubah sampel paralel menjadi urutan bit I2S lengkap dengan `BCLK`, `WS`, dan *delay bit* sesuai spesifikasi Philips.

Listing III.1 menampilkan modul `i2s.v` secara utuh sampai `endmodule`. Penyajian penuh dipilih karena ukuran berkas ini masih ringkas dan seluruh isinya relevan untuk menunjukkan peran *wrapper* pada arsitektur IP yang diusulkan.

```

1 module i2s #(
2     parameter integer C_S00_AXI_DATA_WIDTH = 32,
3     parameter integer C_S00_AXI_ADDR_WIDTH = 4
4 ) (
5     input wire audio_48_clk,
6     input wire audio_44_clk,
7     output wire i2s_mclk,
8     output wire i2s_bclk,
9     output wire i2s_ws,
10    output wire i2s_data,
11    input wire s00_axi_aclk,
12    input wire s00_axi_aresetn,
13    input wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_awaddr,
14    input wire [2:0] s00_axi_awprot,
15    input wire s00_axi_awvalid,
16    output wire s00_axi_awready,
17    input wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_wdata,
18    input wire [(C_S00_AXI_DATA_WIDTH/8)-1:0] s00_axi_wstrb,
19    input wire s00_axi_wvalid,
20    output wire s00_axi_wready,
21    output wire [1:0] s00_axi_bresp,
22    output wire s00_axi_bvalid,
23    input wire s00_axi_bready,
24    input wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_araddr,
25    input wire [2:0] s00_axi_arprot,
26    input wire s00_axi_arvalid,
27    output wire s00_axi_arready,
28    output wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_rdata,
29    output wire [1:0] s00_axi_rresp,

```

```

30  output wire          s00_axi_rvalid,
31  input wire          s00_axi_rready
32  );
33
34  i2s_slave_lite_v1_0_S00_AXI #(
35      .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
36      .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
37  ) i2s_slave_lite_v1_0_S00_AXI_inst (
38      .audio_48_clk (audio_48_clk),
39      .audio_44_clk (audio_44_clk),
40      .i2s_mclk     (i2s_mclk),
41      .i2s_bclk     (i2s_bclk),
42      .i2s_ws       (i2s_ws),
43      .i2s_data     (i2s_data),
44      .S_AXI_ACLK   (s00_axi_aclk),
45      .S_AXI_ARESETN (s00_axi_aresetn),
46      .S_AXI_AWADDR (s00_axi_awaddr),
47      .S_AXI_AWPROT (s00_axi_awprot),
48      .S_AXI_AWVALID (s00_axi_awvalid),
49      .S_AXI_AWREADY (s00_axi_awready),
50      .S_AXI_WDATA   (s00_axi_wdata),
51      .S_AXI_WSTRB   (s00_axi_wstrb),
52      .S_AXI_WVALID  (s00_axi_wvalid),
53      .S_AXI_WREADY  (s00_axi_wready),
54      .S_AXI_BRESP   (s00_axi_bresp),
55      .S_AXI_BVALID  (s00_axi_bvalid),
56      .S_AXI_BREADY  (s00_axi_bready),
57      .S_AXI_ARADDR  (s00_axi_araddr),
58      .S_AXI_ARPROT  (s00_axi_arprot),
59      .S_AXI_ARVALID (s00_axi_arvalid),
60      .S_AXI_ARREADY (s00_axi_arready),
61      .S_AXI_RDATA   (s00_axi_rdata),
62      .S_AXI_RRESP   (s00_axi_rresp),
63      .S_AXI_RVALID  (s00_axi_rvalid),
64      .S_AXI_RREADY  (s00_axi_rready)
65  );
66  endmodule

```

Listing III.1. Kode lengkap modul wrapper `i2s.v`.

Dari listing lengkap tersebut terlihat bahwa `i2s.v` tidak memuat logika serializer maupun *register map* secara langsung. Seluruh perilaku inti ditempatkan pada modul AXI

inti. Pemisahan ini bermanfaat karena perubahan pada port tingkat atas atau integrasi IP tidak akan mengganggu logika internal yang menangani register, CDC, dan pembangkitan sinyal I2S.

3.1.3 Register map

Tabel 3.1 merangkum register word-aligned yang digunakan pada implementasi final.

Tabel 3.1. Register map IP I2S AXI4-Lite.

Offset	Nama Register	Fungsi	Packing / Catatan
0x00	DATA_LEFT	Sampel kanal kiri	MSB di bit [SAMPLE_WIDTH-1]
0x04	DATA_RIGHT	Sampel kanal kanan	MSB di bit [SAMPLE_WIDTH-1]
0x08	CONTROL	Konfigurasi peripheral	Lihat Tabel 3.2

Tabel 3.2. Bitfield register CONTROL.

Bit(s)	Field	Default	Fungsi / Keterangan
0	ENABLE	0	1 = output audio aktif, 0 = output nonaktif
1	MUTE	0	1 = data audio dipaksa nol, clock tetap berjalan
2	FS_FAMILY	0	0 = keluarga 48/96 kHz, 1 = keluarga 44,1/88,2 kHz
3	FS_MODE	0	0 = Fs rendah (48/44,1 kHz); 1 = Fs tinggi (96/88,2 kHz)
7:4	RESERVED	0	Cadangan untuk ekstensi di masa depan
12:8	SAMPLE_WIDTH	32	Lebar bit sampel per kanal (1–32); nilai 0 diinterpretasikan sebagai 32
31:13	RESERVED	0	Cadangan

RESERVED	SAMPLE WIDTH	RESV	FSM	FSF	M	EN
----------	-----------------	------	-----	-----	---	----

Gambar 3.3. Bit diagram register CONTROL (bit 31..13 RESERVED, 12..8 SAMPLE_WIDTH, 7..4 RESERVED, 3 FS_MODE, 2 FS_FAMILY, 1 MUTE, 0 ENABLE).

3.1.4 Aturan packing data

- DATA_LEFT: sampel kanal kiri ditulis dengan **MSB di bit [SAMPLE_WIDTH-1]** dari word 32-bit. Bit di atas SAMPLE_WIDTH-1 diabaikan oleh RTL.
- DATA_RIGHT: aturan packing sama dengan DATA_LEFT, untuk sampel kanal kanan.

- Firmware harus selalu menulis **kedua** register setiap periode sampel agar counter CDC RTL bertambah secara konsisten untuk kedua kanal.
- Contoh konversi untuk sampel int16: geser kiri sebesar $(\text{SAMPLE_WIDTH} - 16)$ bit sehingga MSB tepat di bit $[\text{SAMPLE_WIDTH}-1]$, **tanpa masker tambahan**.
- Jika MUTE=1, serializer memaksa keluaran DATA ke nol tanpa mengubah isi register sampel.

Sebagai contoh, jika firmware menghasilkan sampel bertipe `int16_t` bernilai -32768 dan sistem dikonfigurasi dengan `SAMPLE_WIDTH=24`, maka sampel tersebut harus digeser ke kiri 8 bit agar bit tanda berada tepat pada posisi $[23]$. Dengan demikian, RTL dapat langsung mengambil bit $[23:0]$ sebagai data valid untuk slot I2S, sedangkan bit di atasnya diabaikan. Penjelasan ini penting karena kesalahan kecil pada posisi MSB atau penggunaan masker yang tidak tepat dapat mengubah arti data bertanda, meskipun format register pada sisi AXI tetap tampak benar.

Strategi pembentukan `CONTROL` dari sisi perangkat lunak ditunjukkan pada Listing III.2. Cuplikan ini penting karena memperlihatkan hubungan langsung antara variabel state firmware dan bitfield yang telah didefinisikan pada register `CONTROL`.

```

1 #define CTRL_ENABLE (1U << 0)
2 #define CTRL_MUTE (1U << 1)
3 #define CTRL_FS (1U << 2)
4 #define CTRL_FS_MODE (1U << 3)
5 #define CTRL_WIDTH(w) (((uint32_t)(w) & 0x1FU) << 8U)
6
7 static void apply_control(void)
8 {
9     uint32_t ctrl = 0U;
10     if (g_enable) ctrl |= CTRL_ENABLE;
11     if (g_mute) ctrl |= CTRL_MUTE;
12     if (g_fs) ctrl |= CTRL_FS;
13     if (g_fs_mode) ctrl |= CTRL_FS_MODE;
14     ctrl |= CTRL_WIDTH(g_width);
15     Xil_Out32(CONTROL, ctrl);
16 }

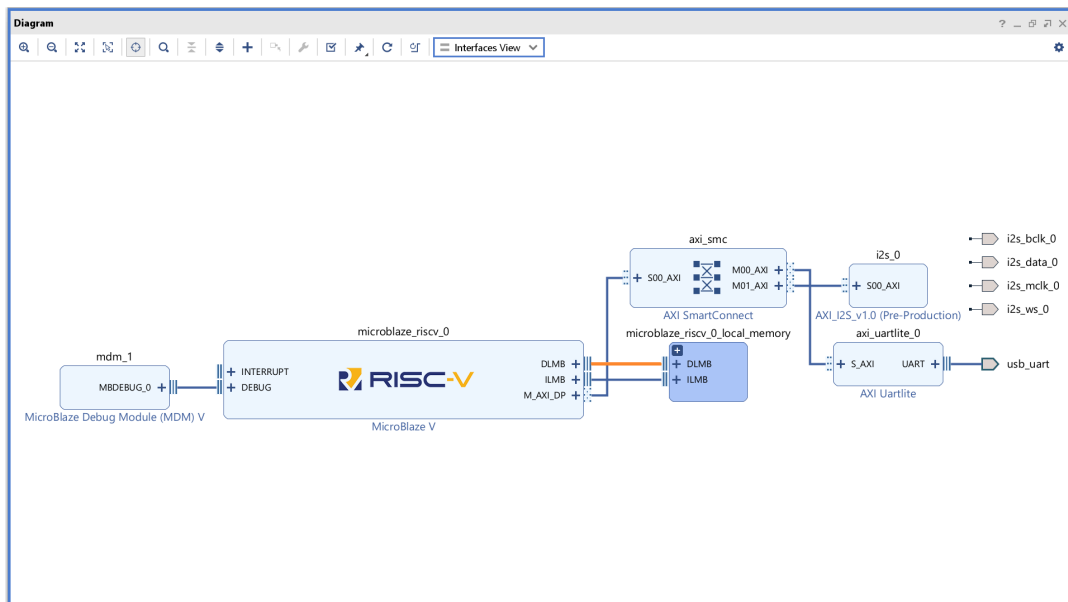
```

Listing III.2. Cuplikan fungsi `apply_control()` pada firmware.

Fungsi tersebut menunjukkan bahwa model kendali peripheral benar-benar bersifat *register-based*. Firmware tidak perlu memanggil prosedur kompleks untuk mengganti mode operasi; cukup menyusun word kontrol 32-bit lalu menuliskannya ke alamat `CONTROL`. Pendekatan ini membuat perilaku sistem lebih mudah ditelusuri saat validasi, karena setiap

perubahan mode dapat dipetakan langsung ke bit tertentu pada register.

3.1.5 Integrasi Vivado Block Design



Gambar 3.4. Integrasi IP I2S pada Block Design.

3.2 Alat dan Bahan

3.2.1 Perangkat keras

3.2.1.1 Board FPGA

Basys3 digunakan sebagai platform implementasi utama [10]. Board ini berbasis FPGA Xilinx Artix-7 dan menyediakan sumber clock 100 MHz, antarmuka USB-UART, serta header PMOD untuk koneksi ke perangkat eksternal.

3.2.1.2 Modul DAC audio

PCM5102A Audio DAC Module digunakan sebagai penerima data I2S [9]. Modul ini mendukung antarmuka audio serial, berbagai *sample rate* hingga 384 kHz, dan keluaran analog stereo untuk verifikasi suara.

3.2.1.3 Perangkat bantu

- **Oscilloscope / logic analyzer / ILA** untuk memverifikasi sinyal BCLK, WS, dan DATA.
- **Speaker atau headphone aktif** untuk uji keluaran audio.
- **Breadboard dan kabel jumper** untuk koneksi Basys3 ke PCM5102A.

3.2.2 Perangkat lunak

- **AMD Vivado Design Suite** untuk RTL design, synthesis, implementation, dan integrasi SoC [11].
- **Xilinx Vitis** untuk pengembangan perangkat lunak uji dalam bahasa C [12].
- **Vivado Simulator** atau simulasi lain bila digunakan untuk verifikasi RTL.
- **LaTeX** untuk penulisan dokumen skripsi.

3.3 Metode yang Digunakan

3.3.1 Metode perancangan hardware

Perancangan perangkat keras dilakukan dengan tahapan berikut:

1. Menentukan spesifikasi I2S TX meliputi format data [1], lebar sampel, mode frekuensi sampling, dan pemetaan register.
2. Mendesain arsitektur modul yang mencakup AXI4-Lite slave [7], register data dan kontrol, pembagi clock, serta serializer I2S.
3. Mengimplementasikan RTL dalam Verilog [3] dengan pemisahan blok sekuensial dan kombinasional secara jelas.
4. Mengintegrasikan peripheral ke dalam Vivado IP Integrator [11] bersama MicroBlaze V, BRAM, clocking, dan peripheral pendukung.

Alur tersebut sejalan dengan pendekatan pengembangan sistem FPGA embedded yang menempatkan definisi spesifikasi, implementasi RTL, integrasi IP, dan validasi perangkat lunak sebagai tahapan yang saling terkait, bukan pekerjaan yang berdiri sendiri [16].

3.3.2 Metode verifikasi dan simulasi

Verifikasi dilakukan pada dua level:

1. **Validasi hardware**, untuk memastikan desain tetap bekerja setelah *synthesis* dan *implementation* pada FPGA nyata.
2. **Verifikasi fungsional RTL**, untuk mereproduksi perilaku sinyal I2S, urutan bit, dan logika DATA/CONTROL secara terkontrol melalui simulasi.

Parameter yang diamati meliputi:

- relasi timing `i2s_bclk`, `i2s_ws`, dan `i2s_data`;
- kontinuitas aliran sampel pada skenario uji;
- akurasi frekuensi sampling nominal rendah dan tinggi;
- keberhasilan keluaran audio pada DAC.

Suatu pengujian dinyatakan lulus apabila empat syarat utama terpenuhi, yaitu: struktur frame terdiri dari 64 pulsa BCLK per sampel stereo, transisi *WS* terjadi satu BCLK sebelum MSB kanal berikutnya, bit tunda setelah transisi *WS* bernilai nol, dan DAC mampu menghasilkan audio yang konsisten dengan mode kanal serta frekuensi yang dipilih. Dengan definisi ini, verifikasi tidak berhenti pada kecocokan waveform semata, tetapi juga menilai apakah perilaku sistem sudah sesuai dengan tujuan operasional peripheral.

Pada alur kerja aktual penelitian ini, verifikasi perilaku dasar terlebih dahulu dilakukan melalui *bitstream bring-up*, pengamatan ILA, dan pengukuran pada perangkat keras. Setelah hubungan timing utama telah teridentifikasi pada FPGA nyata, dibuat *testbench* Vivado/XSim pada proyek `C:\tesis2s\testblock` untuk mereproduksi kondisi *enable*, *mute*, pemilihan *fs_mode*, dan serialisasi sampel kiri-kanan secara lebih formal. Dengan demikian, simulasi pada penelitian ini berfungsi sebagai verifikasi pelengkap yang mendokumentasikan ulang perilaku yang telah diamati pada perangkat keras, bukan sebagai satu-satunya sumber validasi.

Di sisi firmware, pembangkitan sampel uji sekaligus penulisan ke register data ditunjukkan pada Listing III.3. Cuplikan ini menjadi penghubung penting antara teori packing data dan cara perangkat lunak benar-benar memasukkan sampel ke peripheral.

```

1 int32_t s32;
2 if (g_width >= 32U) {
3     s32 = (int32_t)s << 16;
4 } else if (g_width > 16U) {
5     s32 = (int32_t)s << (g_width - 16U);
6 } else if (g_width < 16U) {
7     s32 = (int32_t)s >> (16U - g_width);
8 } else {
9     s32 = (int32_t)s;
10 }
11 uint32_t samp = (uint32_t)s32;
12
13 Xil_Out32(DATA_LEFT, g_left ? samp : 0U);
14 Xil_Out32(DATA_RIGHT, g_right ? samp : 0U);

```

Listing III.3. Cuplikan fungsi `stream_sample()` untuk packing dan penulisan sampel.

Dua hal dapat diamati dari listing tersebut. Pertama, posisi MSB sampel selalu disesuaikan terhadap *g_width*, sehingga format data pada sisi firmware konsisten dengan ekspektasi serializer RTL. Kedua, kedua register `DATA_LEFT` dan `DATA_RIGHT` selalu ditulis pada setiap iterasi, termasuk ketika salah satu kanal di-nol-kan. Keputusan ini sesuai dengan rancangan CDC pada hardware yang melatch pasangan sampel kiri-kanan

secara bersamaan pada batas frame.

3.3.3 Metode integrasi SoC

Integrasi peripheral ke dalam SoC dilakukan melalui alur berikut:

1. **IP Packaging:** mengemas modul I2S sebagai custom IP menggunakan Vivado IP Integrator [11, 24].
2. **Block Design:** menghubungkan MicroBlaze V [6], AXI interconnect [21], BRAM, Clocking Wizard [8], UART, dan IP I2S.
3. **Address Allocation:** menetapkan base address peripheral agar dapat diakses dari Vitis [12].
4. **Constraint Management:** memetakan pin I2S ke konektor board sesuai file `.xdc` [10].

Referensi mengenai *custom IP* penting pada tahap ini karena integrasi peripheral pada Vivado tidak hanya memerlukan RTL yang berfungsi, tetapi juga metadata IP, antarmuka bus yang dikenali tool, dan kompatibilitas dengan mekanisme packaging pada IP Integrator [24]. Oleh sebab itu, keberhasilan integrasi SoC pada penelitian ini bergantung pada dua aspek sekaligus, yaitu kebenaran logika internal peripheral dan ketepatan pengemasannya sebagai IP yang dapat dikonsumsi oleh Block Design.

3.3.4 Metode pengembangan software

Perangkat lunak uji dikembangkan dalam bahasa C pada Vitis dengan fungsi utama:

- menginisialisasi register peripheral I2S;
- menulis sampel audio ke register `DATA_LEFT` dan `DATA_RIGHT`;
- mengaktifkan mode operasi dan memilih frekuensi sampling;
- menyinkronkan waktu penulisan data ke dalam register secara langsung (*programmed I/O*);
- menghasilkan sinyal uji sederhana seperti gelombang sinus atau *square wave*.

3.3.5 Metode pengujian hardware

Pengujian hardware dilakukan dengan langkah berikut:

1. Memprogram FPGA dengan bitstream hasil implementasi.
2. Menghubungkan Basys3 ke PCM5102A.
3. Menjalankan aplikasi Vitis untuk mengirim sampel audio.
4. Mengamati sinyal I2S menggunakan ILA atau osiloskop.

5. Memverifikasi keluaran audio melalui speaker/headphone.

Skenario uji perangkat keras disusun agar mencakup fungsi-fungsi utama peripheral, yakni pengaktifan dan penonaktifan output, pengujian mode MUTE, pemisahan kanal kiri dan kanan, perubahan SAMPLE_WIDTH, serta pemilihan keluarga frekuensi sampling. Setiap skenario diamati pada dua level sekaligus: level sinyal digital menggunakan ILA atau osiloskop untuk memverifikasi ketepatan protokol, dan level audio analog untuk memastikan bahwa konfigurasi register benar-benar menghasilkan perilaku yang diharapkan pada DAC.

3.3.6 Pengukuran dan evaluasi

Metode evaluasi meliputi:

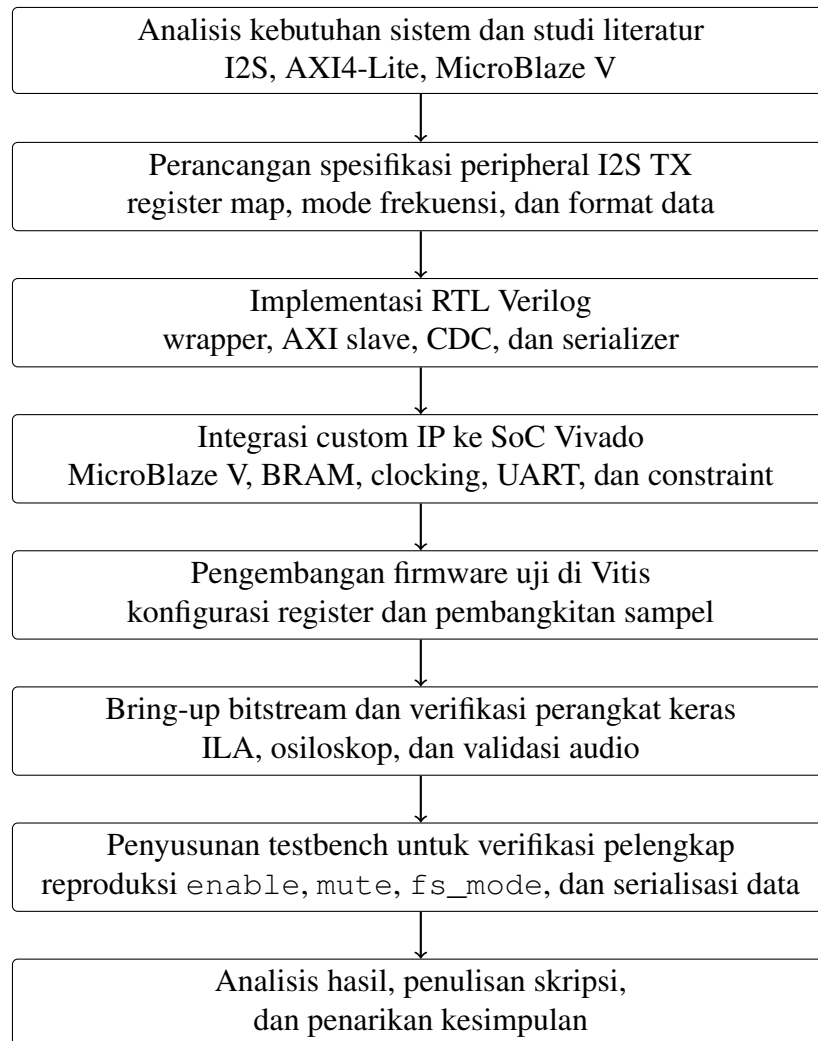
- **Resource utilization:** LUT, FF, dan BRAM dari laporan implementasi.
- **Clock accuracy:** frekuensi aktual hasil Clocking Wizard dan frekuensi terukur sinyal I2S.
- **Fungsionalitas audio:** keberhasilan playback dan kontinuitas sampel yang memadai pada skenario uji utama.

3.4 Alur Tugas Akhir

Penelitian ini mengikuti tahapan berikut:

1. Analisis kebutuhan sistem audio digital dan studi literatur tentang I2S, AXI4-Lite, dan MicroBlaze V.
2. Perancangan spesifikasi peripheral I2S TX.
3. Implementasi RTL Verilog dan verifikasi awal.
4. Integrasi ke dalam SoC pada Vivado Block Design.
5. Pengembangan perangkat lunak uji di Vitis.
6. Pengujian hardware dengan ILA, pengukuran sinyal, dan validasi audio.
7. Analisis hasil, penyusunan dokumen, dan penarikan kesimpulan.

Agar alur kerja lebih mudah dipahami daripada tabel durasi mingguan, tahapan penelitian dirangkum kembali dalam bentuk diagram alir pada Gambar 3.5. Bentuk ini lebih sesuai untuk menunjukkan hubungan logis antar-tahap, termasuk fakta bahwa testbench disusun setelah verifikasi dasar pada perangkat keras telah dilakukan.



Gambar 3.5. Diagram alir tahapan penelitian tugas akhir.

BAB IV

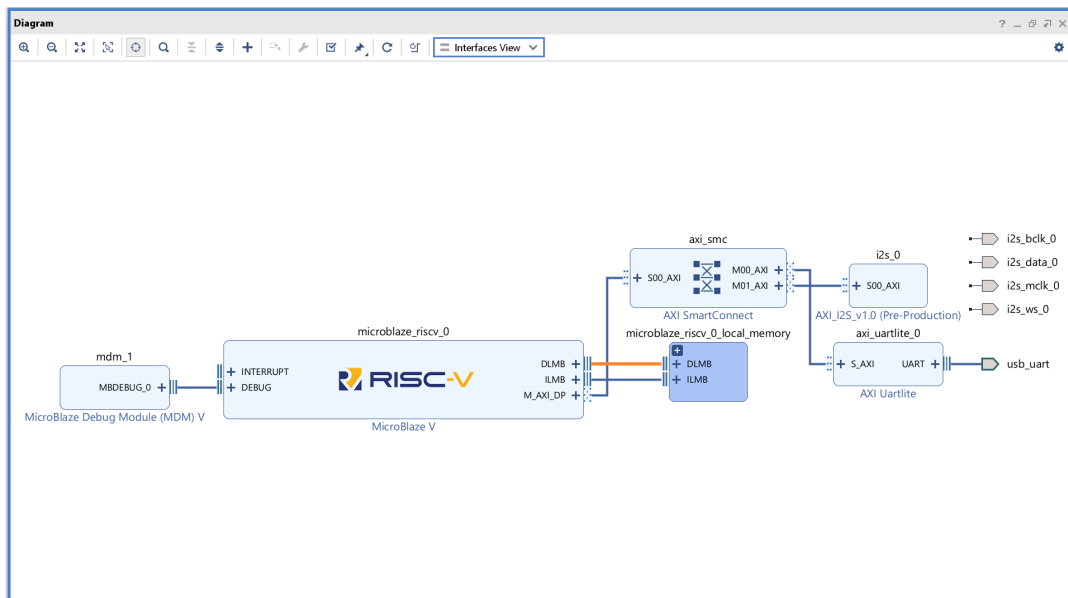
HASIL DAN PEMBAHASAN

Bab ini menyajikan **implementasi** IP I2S TX AXI4-Lite, integrasi pada Basys3, hasil pengujian sinyal dan audio, serta analisis dan perbaikan bug yang ditemukan selama validasi perangkat keras.

4.1 Implementasi RTL dan Integrasi Vivado

RTL direalisasikan dalam dua berkas Verilog: `i2s.v` sebagai *top-level wrapper* dan `i2s_slave_lite_v1_0_S00_AXI.v` sebagai modul utama yang mencakup seluruh logika AXI4-Lite, CDC, dan serializer I2S sesuai spesifikasi Philips [1]. Integrasi dilakukan melalui Vivado IP Integrator (*Block Design*) [11]; pin I2S dipetakan pada file `.xdc`.

Pemisahan antara `i2s.v` dan `i2s_slave_lite_v1_0_S00_AXI.v` dilakukan untuk menjaga tanggung jawab modul tetap jelas. Berkas `i2s.v` berfungsi sebagai pembungkus antarmuka tingkat atas dan penghubung ke sistem Vivado, sedangkan berkas utama menangani detail perilaku peripheral seperti transaksi AXI, sinkronisasi antar-domain clock, pemilihan clock audio, dan pembangkitan data serial I2S. Pemisahan ini memudahkan proses debugging karena permasalahan integrasi sistem dapat dibedakan dari permasalahan logika internal peripheral.



Gambar 4.6. Block Design Vivado: MicroBlaze V terhubung ke IP I2S melalui AXI SmartConnect.

4.1.1 Pemetaan pin (constraint)

Pin I2S dihubungkan ke header PMOD JA pada Basys3 melalui file `cnstr.xdc` sebagai berikut:

```
set_property -dict {PACKAGE_PIN J1 IOSTANDARD LVCMOS33} \
    [get_ports i2s_mclk_0]
set_property -dict {PACKAGE_PIN L2 IOSTANDARD LVCMOS33} \
    [get_ports i2s_bclk_0]
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} \
    [get_ports i2s_ws_0]
set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} \
    [get_ports i2s_data_0]

## Configuration options for Basys3
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

Selain pemetaan pin, file constraint juga mendefinisikan hubungan asinkron antara domain clock AXI (100 MHz) dan domain clock audio (24 MHz) untuk memastikan analisis timing yang benar:

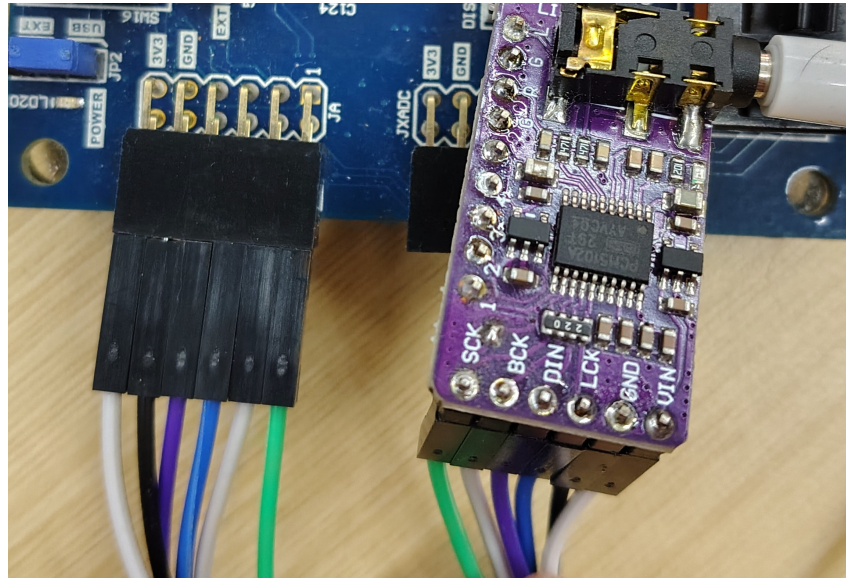
```
set_clock_groups -asynchronous \
    -group [get_clocks clk_out1_bdesign_clock_core_clk_wiz_0_0] \
    -group [get_clocks clk_out2_bdesign_clock_core_clk_wiz_0_0] \
    -group [get_clocks clk_out3_bdesign_clock_core_clk_wiz_0_0] \
    -group [get_clocks -quiet internal_mclk_from_48] \
    -group [get_clocks -quiet internal_mclk_from_44]

set_false_path -from [get_cells -hierarchical *slv_reg2_reg[2]] \
    -to [get_cells -hierarchical *mclk_mux_inst/q*_d1_reg*]
```

Ketiga sinyal utama I2S (BCLK, WS/LRCK, DATA) dan MCLK terhubung langsung ke PMOD JA pin 1–4, yang kemudian disambungkan ke modul PCM5102A menggunakan kabel jumper.

Tahap pemetaan pin mempunyai peran penting karena kesalahan kecil pada penugasan pin atau standar I/O dapat menghasilkan gejala yang mirip dengan kegagalan RTL, misalnya tidak adanya keluaran audio atau bentuk gelombang yang tampak salah. Oleh

sebab itu, validasi pin, tegangan LVCMOS33, dan hubungan antar-domain clock pada file constraint merupakan bagian integral dari keberhasilan implementasi, bukan sekadar konfigurasi administratif.



Gambar 4.7. Wiring PMOD JA ke modul PCM5102A.

4.1.2 Detail logika RTL (serializer dan CDC)

Logika internal IP I2S TX dirancang untuk menangani dua domain clock yang berbeda secara aman. Bagian ini merinci mekanisme tersebut.

Sebelum data mencapai serializer, register sampel dan kontrol terlebih dahulu diisi melalui jalur AXI4-Lite. Cuplikan logika tulis register ditunjukkan pada Listing IV.1. Listing ini penting karena menunjukkan bahwa alamat word-aligned 0×00 , 0×04 , dan 0×08 benar-benar dipetakan ke `slv_reg0`, `slv_reg1`, dan `slv_reg2`.

```

1 always @(posedge S_AXI_ACLK) begin
2     if (S_AXI_ARESETN == 1'b0) begin
3         slv_reg0 <= 0;
4         slv_reg1 <= 0;
5         slv_reg2 <= 0;
6         slv_reg3 <= 0;
7     end else begin
8         if (S_AXI_WVALID) begin
9             case ((S_AXI_AWVALID) ?
10                 S_AXI_AWADDR[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] :
11                 axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
12                 REG_DATA_LEFT:
13                     ...
14                 REG_DATA_RIGHT:

```

```

15      ...
16      REG_CONTROL:
17      ...
18      REG_RESERVED:
19      ...
20      endcase
21  end
22 end
23 end

```

Listing IV.1. Cuplikan logika write register AXI4-Lite.

Meskipun sebagian rincian byte-enable disingkat dengan elipsis pada listing, struktur utamanya tetap terlihat jelas. Setiap transaksi tulis yang valid akan memilih salah satu register berdasarkan alamat AXI, lalu hanya byte yang ditandai oleh `WSTRB` yang diperbarui. Dengan cara ini, peripheral bersifat kompatibel dengan semantik standar AXI4-Lite dan dapat diakses dari perangkat lunak menggunakan operasi `Xil_Out32()` biasa tanpa memerlukan protokol tambahan di luar register map.

4.1.2.1 Mekanisme Clock Domain Crossing (CDC)

Data audio ditulis oleh MicroBlaze V pada domain `S_AXI_ACLK` (100 MHz), sementara serializer beroperasi pada domain `internal_mclk` (~24 MHz). Untuk mencegah metastabilitas, register `CONTROL`, `DATA_LEFT`, dan `DATA_RIGHT` dilewatkan melalui modul `cdc_sync` yang mengimplementasikan sinkronisasi flip-flop ganda (*two-stage synchronizer*) [18]. Selain itu, latching sampel ke register bayangan (*shadow register*) dilakukan tepat pada batas frame stereo (saat `bit_count` kembali ke nol) untuk menjamin integritas data antar kanal.

Implementasi sinkronisasi tersebut ditunjukkan secara ringkas pada Listing IV.2. Cuplikan ini memperlihatkan bahwa bus multi-bit tidak digunakan secara langsung pada domain tujuan, tetapi dilewatkan melalui dua tahap register berurutan.

```

1 module cdc_sync #(
2     parameter integer WIDTH = 32
3 ) (
4     input wire      dest_clk,
5     input wire [WIDTH-1:0] src_in,
6     output wire [WIDTH-1:0] dest_out
7 );
8     (* ASYNC_REG = "TRUE" *) reg [WIDTH-1:0] stage1_q;
9     (* ASYNC_REG = "TRUE" *) reg [WIDTH-1:0] stage2_q;
10

```

```

11  always @(posedge dest_clk) begin
12      stage1_q <= src_in;
13      stage2_q <= stage1_q;
14  end
15
16  assign dest_out = stage2_q;
17 endmodule

```

Listing IV.2. Cuplikan modul `cdc_sync` pada RTL.

Secara praktis, dua tahap register tersebut memberi waktu agar sinyal yang mungkin metastabil pada tahap pertama menjadi stabil sebelum digunakan pada logika domain audio. Pada desain ini, sinkronisasi semacam itu cukup memadai karena yang dipindahkan antar-domain bukan *stream* kontinu berkecepatan tinggi, melainkan status kontrol dan data register yang diperbarui secara diskrit oleh CPU.

4.1.2.2 Logika serializer Philips I2S

Serializer diimplementasikan sebagai mesin state sederhana yang dipicu oleh `bclk_en_w`. Fungsi `i2s_next_serial_bit` menentukan bit mana yang harus dikirim berdasarkan nilai `bit_count_q` (0–63):

- **Bit 0 dan 32:** Merupakan bit tunda (*one-bit delay*) sesuai standar Philips; output dipaksa nol.
- **Bit 1–31 dan 33–63:** Mengirimkan bit sampel dari MSB ke LSB. Jika `SAMPLE_WIDTH` kurang dari 32, sisa bit dalam slot akan diisi nol (*zero padding*).

Sinyal `WS` (Word Select) diturunkan langsung dari bit ke-6 dari `bit_count_q`, sehingga otomatis bernilai rendah untuk 32 bit pertama (kiri) dan tinggi untuk 32 bit berikutnya (kanan). Data diperbarui pada tepi turun `BCLK` dan dipertahankan stabil saat tepi naik `BCLK` untuk menjamin waktu *setup* dan *hold* pada sisi DAC. Cuplikan logika pemilihan bit serial ditunjukkan pada Listing IV.3. Struktur ini menjelaskan secara eksplisit bagaimana aturan *one-bit delay*, pengiriman MSB lebih dulu, dan *zero padding* direalisasikan pada RTL.

```

1  case (bit_count_q)
2      6'd0, 6'd32: begin
3          i2s_next_serial_bit = 1'b0;
4      end
5      default: begin
6          if (bit_count_q < 6'd32) begin
7              if ((bit_count_q - 1) < sample_width_q)
8                  i2s_next_serial_bit = sample_left_q[sample_width_q - bit_count_q];

```

```

9      else
10         i2s_next_serial_bit = 1'b0;
11     end else begin
12         if ((bit_count_q - 6'd33) < sample_width_q)
13             i2s_next_serial_bit = sample_right_q[sample_width_q - (bit_count_q - 6'd32)];
14         else
15             i2s_next_serial_bit = 1'b0;
16         end
17     end
18 endcase

```

Listing IV.3. Cuplikan logika serializer I2S.

Pada bit ke-0 dan ke-32, keluaran dipaksa nol untuk memenuhi *delay bit* setelah transisi WS. Setelah itu, indeks bit sampel dihitung mundur dari MSB menuju LSB. Jika `sample_width_q` lebih kecil dari panjang slot 32-bit, logika secara otomatis mengisi sisa slot dengan nol. Dengan demikian, satu blok serializer yang sama dapat melayani mode 16, 24, maupun 32 bit tanpa mengubah struktur frame stereo. Dengan mekanisme tersebut, perilaku serializer dapat dipandang sebagai rantai proses yang deterministik: sampel baru dari domain AXI tidak langsung dikirim ke jalur serial, tetapi ditahan hingga frame yang sedang berjalan selesai. Strategi ini mencegah terjadinya pergantian nilai di tengah slot yang dapat menyebabkan sebagian bit kanal kiri berasal dari sampel lama dan sebagian bit berikutnya berasal dari sampel baru. Selain itu, kombinasi `ENABLE`, `MUTE`, dan `SAMPLE_WIDTH` memungkinkan satu arsitektur serializer yang sama dipakai untuk beberapa mode operasi tanpa perlu mengubah struktur dasar frame 64-bit.

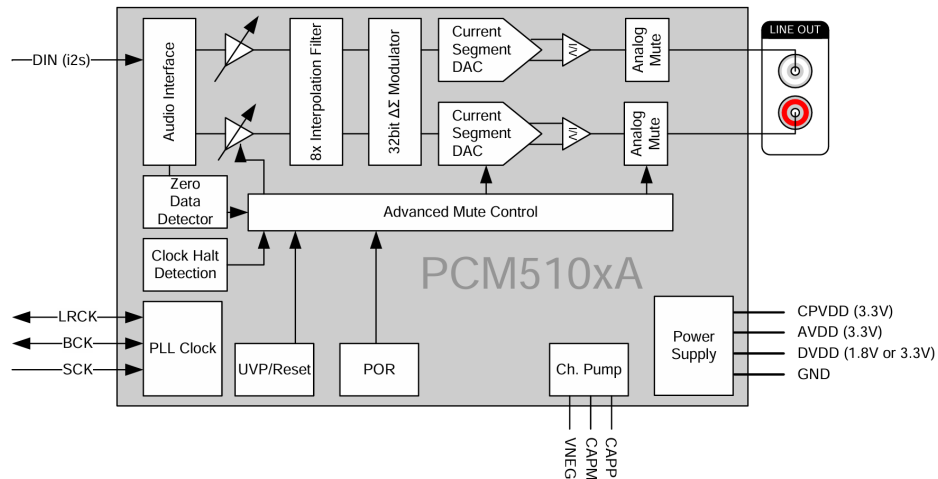
4.1.3 Diagram modul PCM5102A

Modul DAC PCM5102A yang digunakan pada pengembangan sistem ditunjukkan pada dokumentasi Texas Instruments[9]. Gambar resmi dari datasheet dapat dimasukkan secara manual pada bagian ini sesuai kebutuhan penulisan akhir.

4.2 Konfigurasi Clocking Wizard

Clocking Wizard [8] dikonfigurasi menghasilkan tiga keluaran dari satu osilator 100 MHz:

1. `clk_out1` = 100 MHz, digunakan sebagai AXI ACLK dan clock CPU MicroBlaze V.
2. `clk_out2` = 24,576 MHz (aktual 24,59677 MHz), digunakan sebagai `audio_48_clk` untuk keluarga 48/96 kHz.



Gambar 4.8. Diagram blok fungsional PCM5102A dari dokumentasi Texas Instruments.

3. $\text{clk_out3} = 22,579 \text{ MHz}$ (aktual $22,42647 \text{ MHz}$), digunakan sebagai audio_44_clk untuk keluarga $44,1/88,2 \text{ kHz}$.

Tabel 4.1. Konfigurasi clock hasil Clocking Wizard.

Clock	Fungsi / Port	Target	Aktual
clk_out1	AXI ACLK / CPU	100,000 MHz	100,00000 MHz
clk_out2	audio_48_clk	24,576 MHz	24,59677 MHz
clk_out3	audio_44_clk	22,579 MHz	22,42647 MHz

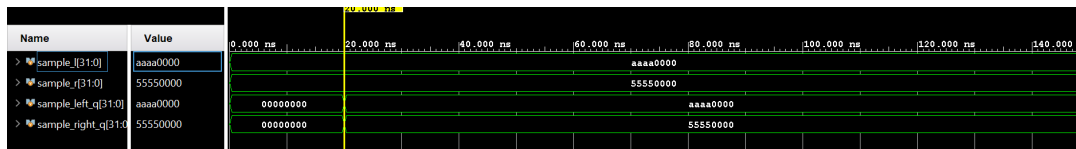
IP memilih antara kedua clock audio menggunakan primitif `BUFGMUX` di dalam RTL, dikontrol oleh bit `FS_FAMILY` pada register `CONTROL` melalui sinkronisasi dua-flip-flop [18]. Perbedaan antara frekuensi *requested* dan *actual* merupakan konsekuensi alami dari keterbatasan rasio pembangkitan MMCM pada FPGA, sehingga tidak dapat dianggap sebagai kesalahan implementasi. Bagi penelitian ini, yang lebih penting adalah apakah selisih tersebut masih cukup kecil sehingga keluaran audio tetap berada pada rentang yang dapat diterima untuk validasi fungsional. Karena penyimpangannya hanya sebagian kecil persen, clock hasil Clocking Wizard masih memadai untuk membuktikan ketepatan arsitektur peripheral dan logika pembagi BCLK.

4.3 Pengujian Sinyal dengan ILA

Integrated Logic Analyzer (ILA) [13] dipasang pada sinyal i2s_bclk , i2s_ws , dan i2s_data untuk memverifikasi timing Philips I2S [1]. Trigger ditempatkan pada tepi turun i2s_ws untuk menangkap awal frame.

Verifikasi ILA mengkonfirmasi:

- Satu frame stereo terdiri dari tepat 64 periode BCLK (32 kiri + 32 kanan).
- WS beralih satu BCLK *sebelum* MSB data, sesuai spesifikasi Philips.



Gambar 4.9. Tangkapan ILA: BCLK, WS, dan DATA – satu frame stereo 64 BCLK.

- Bit pertama setelah setiap peralihan WS selalu bernilai 0 (*delay bit* wajib).
- BCLK = 3,0746 MHz dan WS = 48,03 kHz (konsisten dengan clock aktual 24,597 MHz dan divider 8).

Temuan ini penting karena setiap butir di atas memverifikasi aspek yang berbeda dari desain. Jumlah 64 periode BCLK membuktikan struktur slot 32-bit per kanal, transisi WS memastikan kompatibilitas terhadap format Philips I2S, sedangkan keberadaan *delay bit* menunjukkan bahwa serializer tidak sekadar menggeser data mentah, tetapi benar-benar mengikuti aturan protokol. Kesesuaian antara hasil pengukuran dan perhitungan teoritis juga memperkuat kesimpulan bahwa jalur clock, divider, dan logika frame bekerja secara konsisten.

4.3.1 Verifikasi tambahan menggunakan testbench

Setelah perilaku dasar IP terverifikasi pada perangkat keras nyata, dilakukan verifikasi tambahan menggunakan *testbench* pada proyek C:\tesi2s\testblock, khususnya berkas i2s_tb.v. Testbench ini tidak dimaksudkan untuk menggantikan validasi hardware, melainkan untuk mereproduksi ulang urutan sinyal pada lingkungan yang terkontrol sehingga perubahan status *enable*, *mute*, dan *fs_mode* dapat diamati ulang tanpa memerlukan pemrograman ulang FPGA setiap kali pengujian.

Secara fungsional, testbench memberi stimulus sederhana: sampel kiri 0x0000AAAA, sampel kanan 0x00005555, aktivasi *enable* setelah jeda awal, pengaktifan dan penonaktifan *mute*, serta perubahan *fs_mode* dari mode rendah ke mode tinggi. Rangkaian ini cukup untuk mengonfirmasi bahwa pola serialisasi, perubahan laju BCLK, dan relasi kanal kiri-kanan pada simulasi konsisten dengan hasil pengamatan ILA pada FPGA. Dengan demikian, testbench berperan sebagai alat dokumentasi dan konfirmasi formal terhadap perilaku yang sebelumnya telah dibuktikan pada implementasi nyata.

4.4 Analisis Frekuensi Sampling

Bagian ini menganalisis bagaimana frekuensi sampling terbentuk dari clock audio yang dihasilkan *Clocking Wizard*. Alurnya dapat dibaca secara berurutan sebagai *audio_clk* → pembagi internal → BCLK → F_s . Dengan cara pandang ini, pembaca dapat melihat bahwa frekuensi sampling bukan nilai yang muncul secara terpisah, melainkan konsekuensi langsung dari clock sumber dan konfigurasi divider pada IP.

Pada implementasi ini, BCLK dibangkitkan dari `audio_clk` melalui pembagi internal. Hubungannya dinyatakan sebagai berikut:

$$\text{BCLK} = \frac{\text{MCLK}}{\text{Divider}} \quad (4-1)$$

Setelah BCLK diperoleh, frekuensi sampling dihitung dari jumlah bit dalam satu frame stereo. Karena format yang digunakan adalah 32 bit per kanal untuk dua kanal, maka satu frame berisi 64 siklus BCLK. Oleh sebab itu:

$$F_s = \frac{\text{BCLK}}{64} \quad (4-2)$$

Untuk keluarga frekuensi 48 kHz, clock audio aktual yang digunakan pada pengujian adalah sekitar 24,597 MHz. Berdasarkan relasi tersebut, hasil perhitungan untuk masing-masing mode ditunjukkan pada Tabel 4.2.

Tabel 4.2. Hasil pengujian clock I2S pada keluarga frekuensi 48 kHz.

Mode	MCLK	Divider	BCLK	F_s
FS_MODE = 0	24,597 MHz	÷8	3,0746 MHz	48,03 kHz
FS_MODE = 1	24,597 MHz	÷4	6,1493 MHz	96,08 kHz

Tabel 4.2 memperlihatkan hubungan yang intuitif. Ketika `FS_MODE` = 0, divider bernilai 8 sehingga BCLK turun menjadi sekitar 3,0746 MHz. Karena satu sampel stereo membutuhkan 64 siklus BCLK, frekuensi sampling yang dihasilkan menjadi sekitar 48,03 kHz. Ketika `FS_MODE` = 1, divider diperkecil menjadi 4, sehingga BCLK naik hampir dua kali lipat menjadi 6,1493 MHz. Akibatnya, F_s juga meningkat hampir dua kali lipat menjadi 96,08 kHz.

Dari hasil tersebut dapat dilihat bahwa perubahan `FS_MODE` bekerja sesuai tujuan rancangan: mode pertama menghasilkan keluarga 48 kHz, sedangkan mode kedua menghasilkan keluarga 96 kHz. Selisih kecil terhadap nilai nominal berasal dari frekuensi clock aktual MMCM yang sedikit berbeda dari target ideal 24,576 MHz. Dengan kata lain, penyimpangan ini bersifat sistematis dan dapat diprediksi dari clock sumber, bukan disebabkan oleh kesalahan logika serializer atau bug pada RTL. Untuk tujuan validasi peripheral, hasil ini menunjukkan bahwa jalur clock dan mekanisme pembagi bekerja secara konsisten serta menghasilkan frekuensi sampling yang sangat dekat dengan target desain.

Perbaikan dari sisi firmware yang memungkinkan akurasi ini dipertahankan pada saat streaming ditunjukkan pada Listing IV.4. Fungsi `rdcycle64()` digunakan untuk membentuk basis waktu absolut pada level siklus CPU.

```

1 static inline uint64_t rdcycle64(void)
2 {
3     uint32_t lo, hi, hi2;
4     do {
5         __asm__ volatile ("rdcycleh %0" : "=r"(hi));
6         __asm__ volatile ("rdcycle %0" : "=r"(lo));
7         __asm__ volatile ("rdcycleh %0" : "=r"(hi2));
8     } while (hi != hi2);
9     return ((uint64_t)hi << 32) | lo;
10 }
11
12 uint64_t t_deadline = rdcycle64();
13 while (1) {
14     uint64_t period_cycles =
15         (uint64_t)XPAR_CPU_CORE_CLOCK_FREQ_HZ / current_sample_rate();
16     stream_sample();
17     t_deadline += period_cycles;
18     while (rdcycle64() < t_deadline) {}
19 }

```

Listing IV.4. Cuplikan pembacaan `mcycle` 64-bit dan loop berbasis deadline.

Berbeda dari pendekatan `usleep()`, loop berbasis deadline tidak menebak overhead tiap iterasi. Waktu target selalu dimajukan satu periode sampel secara absolut, lalu CPU menunggu hingga counter mencapai nilai tersebut. Karena itu, error waktu tidak terakumulasi dari satu iterasi ke iterasi berikutnya. Inilah alasan utama mengapa frekuensi hasil perbaikan menjadi jauh lebih dekat dengan nilai teoritis.

4.5 Identifikasi dan Perbaikan Bug Firmware

Selama validasi perangkat keras ditemukan empat gejala tidak benar yang semuanya bersumber dari **kode firmware** (`helloworld.c`), bukan dari RTL. Berikut adalah analisis dan solusi untuk masing-masing bug.

4.5.1 Bug 1: Masker Merusak Bit Tanda Sampel

4.5.1.1 Gejala

Amplitudo gelombang kanal kiri dan kanan tidak sama meskipun nilai yang dikirim identik. Waveform tampak seperti *half-wave rectification* pada salah satu kanal.

4.5.1.2 Penyebab

RTL mengharapkan MSB sampel berada di bit `[sample_width-1]` dari word 32-bit. Kode lama menerapkan masker setelah geseran:

```
1 uint32_t mask = (1U << g_width) - 1U;    // untuk 24-bit: 0
   x00FFFFFF
2 uint32_t samp = (uint32_t)s32 & mask;    // memotong bit [31:24]
```

Sampel negatif int16 setelah digeser kiri 8 bit memiliki bit tanda di posisi `[31:24]`. Masker memotong bit-bit ini sehingga nilai negatif menjadi positif – gelombang sinus terdistorsi.

4.5.1.3 Solusi

Masker dihapus. Geseran disesuaikan agar MSB int16 tepat di bit `[g_width-1]`:

```
1 int32_t s32;
2 if (g_width >= 32U) s32 = (int32_t)s << 16;
3 else if (g_width > 16U) s32 = (int32_t)s << (g_width - 16U);
4 else if (g_width < 16U) s32 = (int32_t)s >> (16U - g_width);
5 else s32 = (int32_t)s;
6 uint32_t samp = (uint32_t)s32;    // tanpa mask
```

RTL hanya membaca bit `[sample_width-1:0]` dari register data; bit di atasnya diabaikan, sehingga tidak diperlukan masker pada sisi firmware.

4.5.2 Bug 2: Mode 32-bit Kehilangan 1 Bit Resolusi

4.5.2.1 Gejala

Saat `SAMPLE_WIDTH=32`, amplitudo gelombang separuh dari yang diharapkan; sampel full-scale positif (32767) tidak menghasilkan output maksimum.

4.5.2.2 Penyebab

Untuk `g_width=32`, geseran kiri 16 menghasilkan `0x7FFF0000` untuk sampel `0x7FFF`. Bit 31 dari word ini adalah 0 (bukan 1). Serializer RTL pada posisi pertama membaca `sample[31]` sebagai MSB, sehingga seluruh nilai tergeser satu bit ke bawah – setara dengan kehilangan satu bit resolusi.

4.5.2.3 Solusi

Sama dengan Bug 1 – perbaikan geseran tanpa masker sudah menyelesaikan masalah ini secara otomatis. `(int32_t)s << 16` menghasilkan penempatan MSB yang benar di bit 31.

4.5.3 Bug 3: Kanal Kanan Mute saat Mode Right-Only

4.5.3.1 Gejala

Menekan R (Right only): kedua kanal menjadi mute atau tidak terdengar sama sekali.

4.5.3.2 Penyebab

Gejala ini merupakan konsekuensi dari Bug 1. Pada mode `g_left=0, g_right=1`, firmware menulis nol ke `DATA_LEFT` dan `samp` ke `DATA_RIGHT`. Karena `samp` sudah rusak akibat masker, output audio tidak lagi mencerminkan gelombang sinus yang benar. Pada frekuensi tertentu, *DC offset* yang terbentuk bahkan dapat diinterpretasikan sebagai tidak ada suara oleh DAC.

4.5.3.3 Solusi

Perbaiki Bug 1 secara langsung menyelesaikan gejala ini. Firmware sudah benar dalam menulis **kedua** register setiap iterasi (baik yang aktif maupun yang bernilai nol), sehingga CDC counter RTL selalu bertambah secara konsisten.

4.5.4 Bug 4: Frekuensi Audio Lebih Rendah dari Target

4.5.4.1 Gejala

Frekuensi terukur pada osiloskop atau spektrum audio lebih rendah dari 1 kHz, 2 kHz, dan 5 kHz yang dipilih. Selisih semakin besar pada frekuensi lebih tinggi.

4.5.4.2 Penyebab

Loop utama menggunakan `usleep(periode - 2)` untuk mengatur laju sampel:

```
1 uint32_t sample_period_us = (1000000U + (sample_rate / 2U)) /  
   sample_rate;  
2 if (sample_period_us > 2) usleep(sample_period_us - 2);
```

Terdapat tiga masalah fundamental:

1. **Koreksi tetap -2 tidak akurat.** Overhead loop (penulisan AXI, polling UART, cache miss) bervariasi dan tidak selalu tepat 2 μ s.
2. **Granularitas `usleep()` terbatas.** Pada lingkungan bare-metal MicroBlaze, resolusi minimum `usleep()` mendekati 1 μ s. Pada 96 kHz (periode 10,4 μ s), koreksi 2 μ s berarti error frekuensi $\approx 20\%$.

3. **Tidak ada akumulasi waktu.** Setiap iterasi mengukur waktu dari nol baru; error per iterasi tidak saling mengkompensasi, melainkan terakumulasi menjadi error frekuensi sistematis.

4.5.4.3 Solusi

Loop diubah menjadi **deadline-based busy-wait** menggunakan *hardware cycle counter* RISC-V (*mcycle* CSR) [20]:

```
1 static inline uint64_t rdcycle64(void)
2 {
3     uint32_t lo, hi, hi2;
4     do {
5         __asm__ volatile ("rdcycleh %0" : "=r"(hi));
6         __asm__ volatile ("rdcycle %0" : "=r"(lo));
7         __asm__ volatile ("rdcycleh %0" : "=r"(hi2));
8     } while (hi != hi2);
9     return ((uint64_t)hi << 32) | lo;
10 }
```

Counter ini berjalan pada frekuensi CPU (100 MHz), memberikan resolusi 10 ns. Loop utama menggunakan deadline absolut:

```
1 uint64_t t_deadline = rdcycle64();
2 while (1) {
3     uint64_t period_cycles = (uint64_t)
4     XPAR_CPU_CORE_CLOCK_FREQ_HZ
5     / current_sample_rate();
6     stream_sample();
7     if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
8         handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
9     t_deadline += period_cycles; // maju satu periode
10    while (rdcycle64() < t_deadline) {} // busy-wait
11 }
```

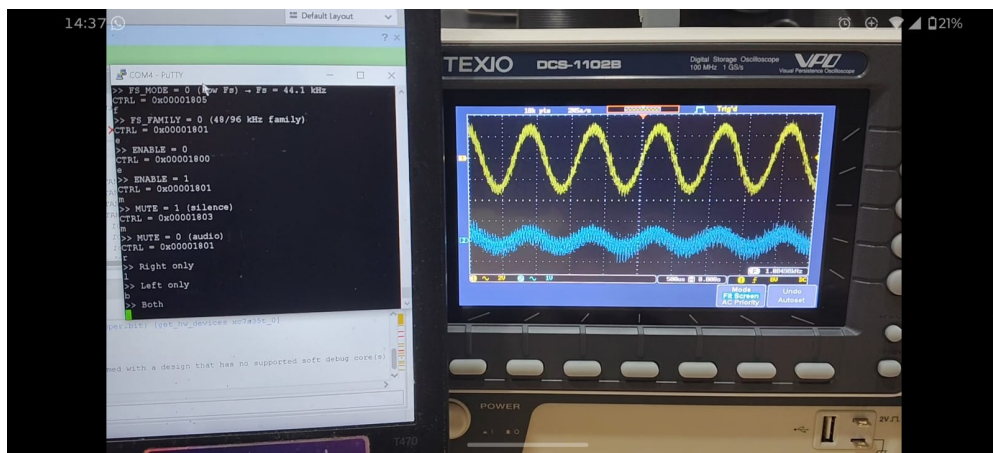
Dengan cara ini, terlepas dari berapa lama operasi dalam iterasi berlangsung, deadline berikutnya selalu berjarak tepat satu periode sampel. Akurasi frekuensi audio menjadi setara dengan akurasi osilator 100 MHz pada board Basys3.

Tabel 4.3. Perbandingan akurasi timing loop sebelum dan sesudah perbaikan.

Properti	usleep() lama	rdcycle64() baru
Resolusi waktu	$\approx 1 \mu s$	10 ns
Kompensasi overhead loop	Tebakan tetap (-2)	Otomatis (deadline absolut)
Akumulasi error	Ya	Tidak
Bekerja di 96 kHz	Tidak ($\approx 20\%$ error)	Ya

4.6 Verifikasi Setelah Perbaikan

Setelah keempat bug diperbaiki dan firmware baru diprogram ke FPGA, pengujian ulang dilakukan:



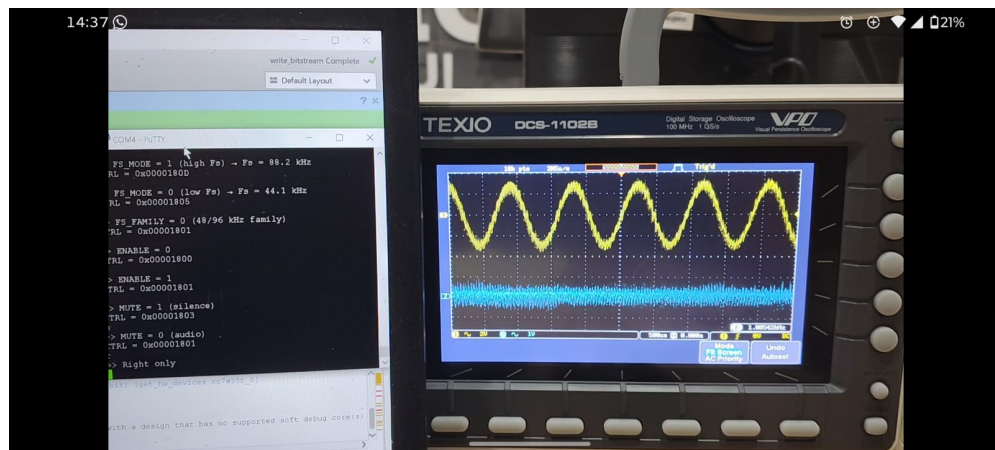
Gambar 4.10. Keluaran audio setelah perbaikan: kanal L dan R memiliki amplitudo dan frekuensi identik.

Hasil verifikasi setelah perbaikan dirangkum pada Tabel 4.4.

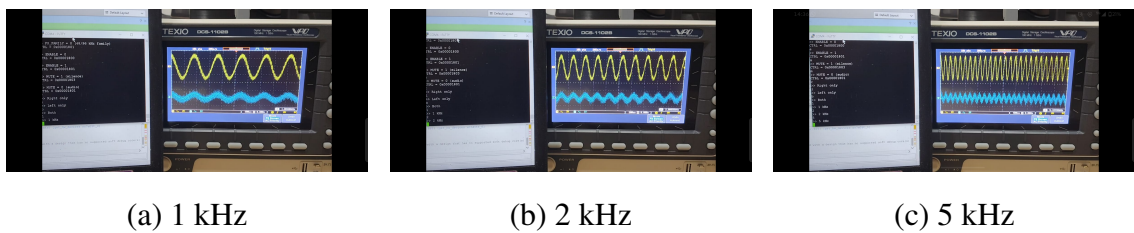
Tabel 4.4 menunjukkan bahwa setelah koreksi firmware, seluruh skenario utama dapat dijalankan tanpa gejala yang sebelumnya muncul. Hal ini memiliki dua makna. Pertama, peripheral telah memenuhi fungsi dasarnya sebagai pemancar I2S yang dapat dikendalikan oleh perangkat lunak. Kedua, proses validasi berhasil memisahkan sumber masalah antara lapisan RTL dan lapisan firmware, sehingga kesimpulan akhir mengenai kualitas desain dapat diberikan dengan keyakinan yang lebih tinggi.

4.7 Pembahasan terhadap Tujuan Penelitian

1. **Tujuan perancangan IP** tercapai: register AXI4-Lite aktif, data kiri-kanan dimuat dari perangkat lunak, serializer menghasilkan format Philips I2S dengan 64 BCLK per frame stereo, dan BUFGMUX menyeleksi clock keluarga 48/44,1 kHz secara benar.
2. **Tujuan integrasi SoC dan Vitis** tercapai: aplikasi bare-metal mengendalikan IP



Gambar 4.11. Mode Right-only setelah perbaikan: kanal kanan aktif, kanal kiri = 0.



Gambar 4.12. Verifikasi frekuensi: osiloskop/ILA tangkapan pada 1 kHz, 2 kHz, dan 5 kHz setelah perbaikan.

Tabel 4.4. Hasil verifikasi firmware setelah perbaikan bug.

Skenario Uji	Hasil yang Diharapkan	Status
Both + 1 kHz	L dan R identik, 1 kHz	✓
Left only + 1 kHz	L aktif, R = 0	✓
Right only + 1 kHz	R aktif, L = 0	✓
Both + 2 kHz	L dan R identik, 2 kHz	✓
Both + 5 kHz	L dan R identik, 5 kHz	✓
MUTE aktif	Kedua kanal = 0, BCLK/WS tetap	✓
SAMPLE_WIDTH = 16	Amplitudo benar, bentuk sinus	✓
SAMPLE_WIDTH = 32	Amplitudo penuh, bentuk sinus	✓

melalui *memory-mapped I/O* (`Xil_Out32/Xil_In32`) untuk inisialisasi, pengisian sampel, dan kontrol mode.

3. **Tujuan verifikasi frekuensi dan audio** tercapai setelah perbaikan empat bug firmware:
 - Bug justifikasi bit (masker merusak tanda) diselesaikan dengan menggeser sampel tanpa masker.
 - Bug mode 32-bit (MSB salah posisi) diselesaikan dengan formula geseran yang diperbaiki.
 - Bug kanal kanan pada mode *right-only* terselesaikan setelah representasi sampel kembali benar.
 - Bug timing loop (`usleep` tidak akumulatif) diselesaikan dengan *deadline-based busy-wait* berbasis `rdcycle64()`.
4. **Penting dicatat** bahwa seluruh perbaikan bersifat *firmware-only* — RTL tidak dimodifikasi. Hal ini mengkonfirmasi bahwa desain RTL sudah benar sejak awal, termasuk mekanisme CDC yang robust dan serializer Philips I2S yang tepat.

Secara keseluruhan, hasil penelitian menunjukkan bahwa pendekatan register-based AXI4-Lite cukup efektif untuk membangun peripheral I2S TX yang sederhana, mudah diintegrasikan, dan mudah dianalisis. Keterbatasan utama sistem ini terletak pada tidak adanya FIFO atau DMA, sehingga kontinuitas sampel masih bergantung pada kemampuan perangkat lunak mempertahankan laju tulis. Meski demikian, untuk tujuan tugas akhir yang menekankan ketepatan protokol, integrasi SoC, dan keterlacakan proses debugging, arsitektur yang dihasilkan sudah memadai dan sesuai dengan sasaran penelitian.

BAB V

TAMBAHAN (OPSIONAL)

Bab ini dapat diisi pembahasan yang memanjang di luar Bab IV, misalnya: perbandingan dengan penelitian terdahulu, atau studi kasus batas frekuensi DAC. Jika tidak diperlukan, subbab berikut dapat dihapus atau diringkas setelah pembimbing menyetujui.

5.1 Perbandingan dengan referensi desain

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.251 ns	Worst Hold Slack (WHS): 0.039 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7290	Total Number of Endpoints: 7290	Total Number of Endpoints: 2637
All user specified timing constraints are met.		

Gambar 5.13. Ringkasan timing desain.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Penelitian ini merancang dan mengimplementasikan **IP core transmitter I2S** [1] dengan antarmuka **AXI4-Lite** [7] pada SoC **MicroBlaze V** [6], board **Basys3** [10], dan keluaran menuju **PCM5102A** [9]. Berdasarkan hasil pada Bab IV, dapat disimpulkan:

1. IP berhasil diintegrasikan sebagai peripheral AXI4-Lite dengan peta register aktif `DATA_LEFT` (`0x00`), `DATA_RIGHT` (`0x04`), dan `CONTROL` (`0x08`). Register `0x0C` disediakan sebagai cadangan dan berhasil digunakan untuk *readback test* AXI yang memverifikasi koneksi bus.
2. Aplikasi Vitis bare-metal dapat mengendalikan IP melalui *memory-mapped I/O* (`Xil_Out32/Xil_In32`) untuk inisialisasi register `CONTROL`, pengisian sampel kanal kiri-kanan, dan verifikasi *readback*.
3. Pengujian ILA [13] mengkonfirmasi sinyal `BCLK` (3,07 MHz), `WS` (48,03 kHz), dan `DATA` mengikuti mekanisme **Philips I2S** [1]: 1-bit delay setelah setiap peralihan `WS`, 64 `BCLK` per frame stereo, dan `MSB-first` per kanal. Nilai aktual sesuai dengan analisis clock Clocking Wizard [8] (24,597 MHz).
4. Validasi audio pada **PCM5102A** [9] berhasil menghasilkan keluaran suara yang dapat didengar. Tiga bug firmware yang ditemukan selama validasi — yaitu *masker bit yang merusak tanda sampel*, *justifikasi sampel 32-bit yang tidak tepat*, dan *timing loop berbasis usleep yang tidak akumulatif* — berhasil diidentifikasi dan diperbaiki seluruhnya di sisi firmware, tanpa perubahan pada RTL.
5. RTL `i2s_slave_lite_v1_0_S00_AXI.v` terbukti benar secara fungsional sejak awal, mencakup mekanisme CDC berbasis write-counter yang robust [18], pemilihan clock melalui `BUFGMUX`, dan serializer Philips I2S yang sesuai spesifikasi [1].

6.2 Saran

1. Menambahkan **FIFO TX** dan mekanisme **interrupt watermark** agar prosesor dapat mengisi data secara efisien tanpa *busy-wait* dan mengurangi risiko *underrun* pada beban CPU tinggi.
2. Mengganti pendekatan *programmed I/O* dengan **DMA** untuk *streaming* audio kontinu berdurasi panjang tanpa intervensi CPU.
3. Menambahkan register **STATUS** yang dapat dibaca oleh firmware untuk memantau

kondisi *underrun*, level buffer, dan kesiapan latch CDC — memudahkan transisi ke driver tingkat sistem operasi.

4. Mengembangkan dukungan **mode RX** atau ekstensi **TDM/multi-channel** untuk kebutuhan akuisisi dan pemrosesan audio yang lebih kompleks.
5. Melakukan karakterisasi lanjutan terhadap akurasi frekuensi audio pada berbagai kondisi suhu dan beban, serta mempertimbangkan penggunaan **PLL audio eksternal** jika akurasi clock $\ll$ 100 ppm diperlukan.
6. Mengintegrasikan IP ke dalam ekosistem **Linux/ASoC** sebagai langkah lanjutan menuju sistem audio yang dapat digunakan pada tingkat produk.

DAFTAR PUSTAKA

- [1] Philips Semiconductors, “I²S bus specification,” Philips Semiconductors, Tech. Rep., Feb. 1986, revision 1. Tersedia: <https://www.nxp.com/docs/en/user-guide/UM11732.pdf>.
- [2] S. M. Trimberger, “Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology,” *Proceedings of the IEEE*, vol. 103, no. 3, pp. 318–331, 2015.
- [3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.
- [4] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, “The Zynq book: Embedded processing with the ARM cortex-A9 on the Xilinx Zynq-7000 all programmable SoC,” *Strathclyde Academic Media*, 2014, iISBN 978-0992978709. Tersedia: <http://www.zynqbook.com>.
- [5] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, “The RISC-V instruction set architecture,” in *Hot Chips 28 Symposium (HCS)*, 2016, pp. 1–52.
- [6] AMD Xilinx, *MicroBlaze Processor Reference Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug984-vivado-microblaze-ref.pdf.
- [7] ARM Limited, “AMBA AXI and ACE protocol specification,” ARM Limited, Tech. Rep. ARM IHI 0022D, 2010, issue D. Tersedia: <https://developer.arm.com/documentation/ih0022/d/>.
- [8] AMD Xilinx, *Clocking Wizard v6.0 Product Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/clk_wiz/v6_0/pg065-clk-wiz.pdf.
- [9] Texas Instruments, “PCM5102A 2-VRMS, 112-dB audio stereo DAC with PLL and 32-bit, 384-kHz PCM interface,” Texas Instruments, Tech. Rep. SLAS764B, 2014, datasheet. Tersedia: <https://www.ti.com/lit/ds/symlink/pcm5102a.pdf>.
- [10] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, Digilent Inc., 2020, tersedia: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>.
- [11] AMD Xilinx, *Vivado Design Suite User Guide: Designing with IP*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug896-vivado-ip.pdf.
- [12] —, *Vitis Unified Software Platform Documentation: Embedded Software Development*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug1400-vitis-embedded.pdf.
- [13] —, *ILA (Integrated Logic Analyzer) v6.2 Product Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/ila/v6_2/pg172-ila.pdf.

- [14] J. Sang, D. Peng, and M. Zhou, “Design and implementation of I2S digital audio interface on FPGA,” *Journal of Physics: Conference Series*, vol. 1601, no. 5, p. 052040, 2020.
- [15] A. Kaviani and D. Lewis, “Embedded logic analyzer and I2S audio interface on low-cost FPGA,” in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2017, pp. 221–224.
- [16] A. Lafosse and D. Gaffe, *Embedded FPGA Systems: Design and Programming with Vivado*. Wiley-ISTE, 2018.
- [17] V. Sklyarov, I. Sklyarova, A. Barkalov, and L. Skliarova, “Hierarchical AXI-bus based SoC for FPGA,” in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 2014, pp. 1–4.
- [18] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011.
- [19] I. Kuon, R. Tessier, and J. Rose, “FPGA architecture: Survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.
- [20] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume I: Unprivileged ISA,” RISC-V Foundation, Tech. Rep. Document Version 20191213, 2019, tersedia: <https://github.com/riscv/riscv-isa-manual/releases/>.
- [21] AMD Xilinx, *AXI Reference Guide*, AMD Xilinx, 2017, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf.
- [22] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.
- [23] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Wiley-IEEE Press, 2008.
- [24] AMD Xilinx, “Designing custom IP for Vivado IP integrator,” AMD Xilinx, Tech. Rep., 2021, tersedia di folder bahan: `custom-ip.pdf`.

LAMPIRAN

L.1 Hasil resource utilization

L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

Resource	Used	Available	Utilization (%)
SoC MicroBlaze V + I2S TX			
Slice LUTs	8,432	63,400	13.30
Slice Registers	6,891	126,800	5.43
Block RAM Tile	28	135	20.74
DSP48 Slices	3	240	1.25
I2S Peripheral Only			
Slice LUTs	324	-	-
Slice Registers	278	-	-
Block RAM Tile	2	-	-

L.1.2 Timing report

Tabel L.2. Hasil analisis timing

Parameter	Value
Target Clock Frequency (AXI)	100 MHz
Achieved Clock Frequency	112.5 MHz
Worst Negative Slack (WNS)	1.234 ns
Total Negative Slack (TNS)	0 ns
Timing Met?	Yes

L.2 Timing diagram

L.2.1 I2S audio transmission timing

Mode uji utama : Fs keluarga 48 kHz
Sample width : 24-bit per kanal (slot 32-bit)
Struktur frame : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri (32 BCLK)

WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:

bit-0 : delay 1-bit (sesuai Philips I2S)
bit-1..24 : data audio (MSB -> LSB)
bit-25..31: zero padding

L.3 Log hasil pengujian

L.3.1 I2S audio playback test

=== I2S AXI4-Lite test harness ===

Base address : 0XXXXXXXXX
Sample rate : 48000 Hz
Sample width : 24 bit
Registers : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08

[TEST] AXI register readback

REG0(0x00) write/read : OK

REG1(0x04) write/read : OK

REG2(0x08) write/read : OK

REG3(0x0C) write/read : OK

[TEST] Stereo sine 440 Hz

I2S streaming active, output teramati pada BCLK/WS/DATA

Audio terverifikasi pada DAC PCM5102A

L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

1. **MicroBlaze V (microblaze_riscv_0)**
2. **AXI Interconnect**
3. **Local Memory / BRAM**
4. **I2S Transmitter (custom IP i2s_v1_0)**
5. **AXI UART Lite**
6. **AXI Interrupt Controller** jika interrupt digunakan
7. **Clocking Wizard**
8. **Processor System Reset**

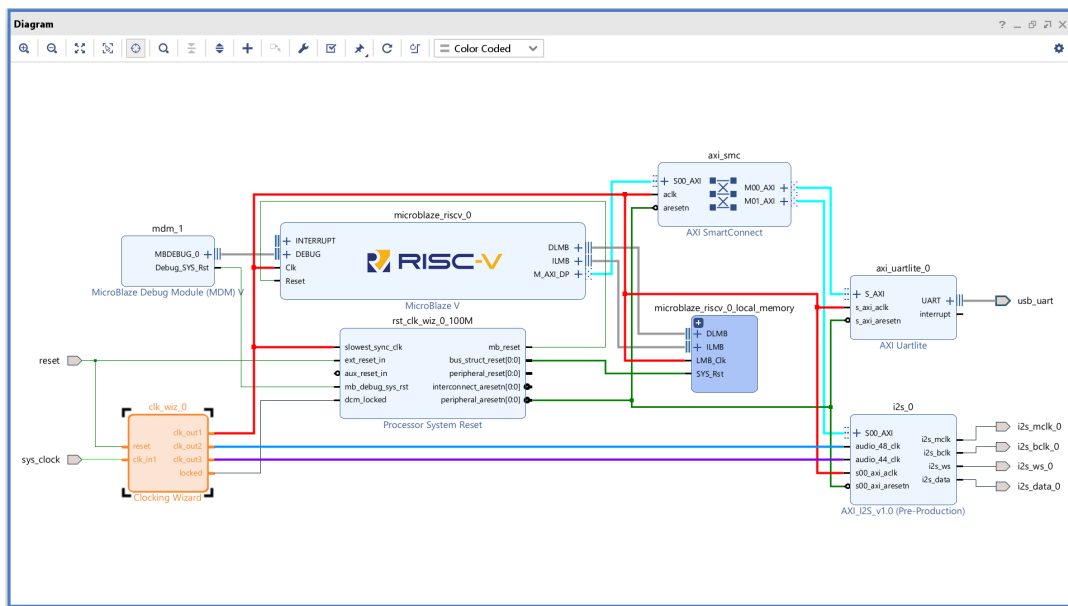
Koneksi utama:

- Semua slave AXI4-Lite terhubung melalui AXI Interconnect.
- Peripheral I2S memakai `audio_clk` untuk membangkitkan `mclk/bclk/ws`.

- Jalur AXI register memakai s00_axi_aclk.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

L.4.1 Diagram Block (Color-coded)

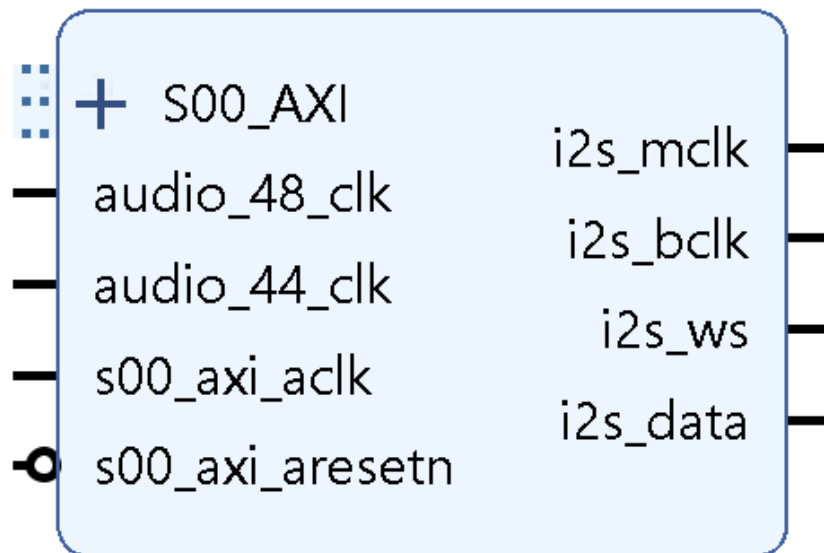
Gambar L.1 menampilkan diagram block warna-warni desain SoC, termasuk domain clock, prosesor, interconnect, dan IP I2S.



Gambar L.1. Diagram block warna-warni arsitektur SoC.

L.4.2 I2S IP Block (Standalone)

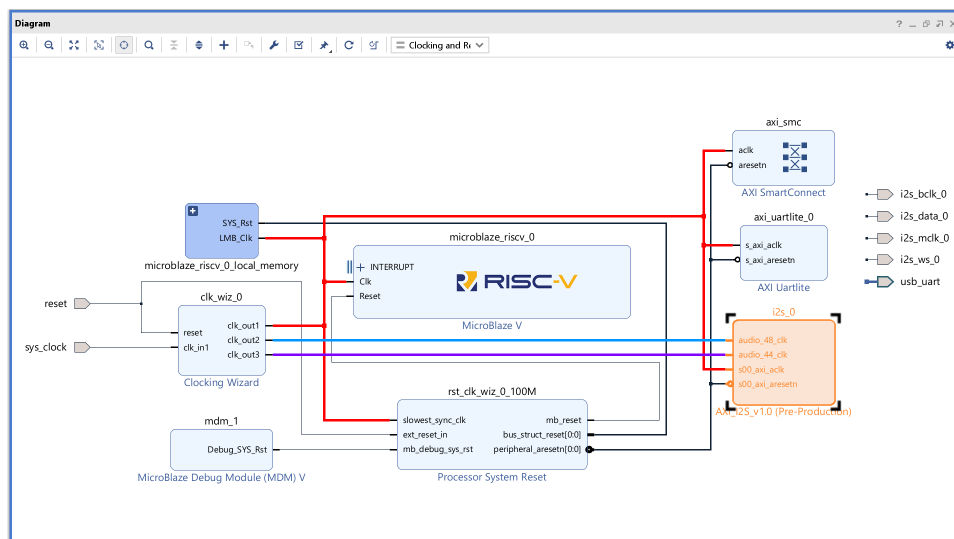
Gambar L.2 memperlihatkan blok I2S IP terpisah beserta port audio clock, sinyal I2S, dan antarmuka AXI4-Lite.



Gambar L.2. Blok IP I2S standalone.

L.4.3 Clocking View

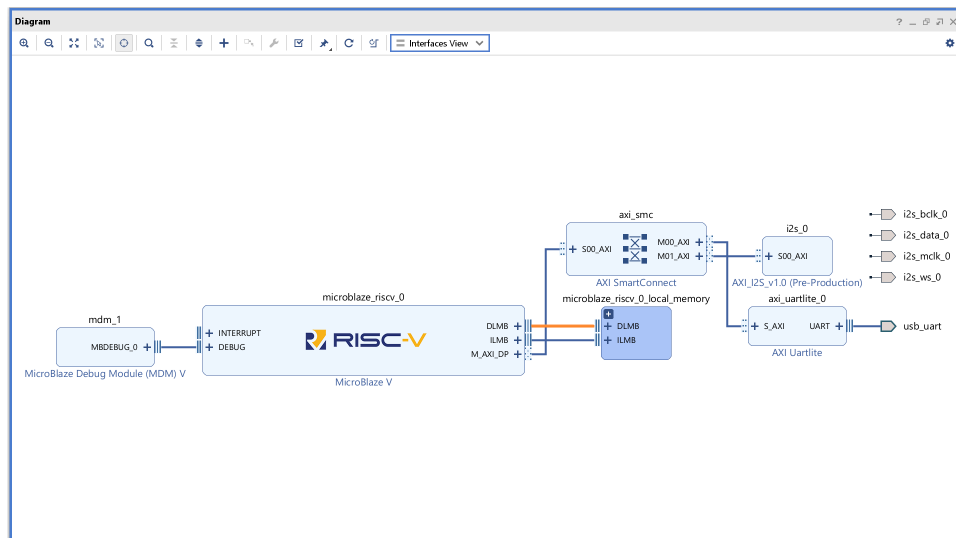
Clocking view menunjukkan Clocking Wizard dan BUFGMUX untuk memilih keluarga 48 kHz atau 44.1 kHz.



Gambar L.3. Clocking view dan jalur pemilihan clock audio.

L.4.4 Interface View

Interface view merinci koneksi pin PMOD, sinyal I2S, dan UART debug.



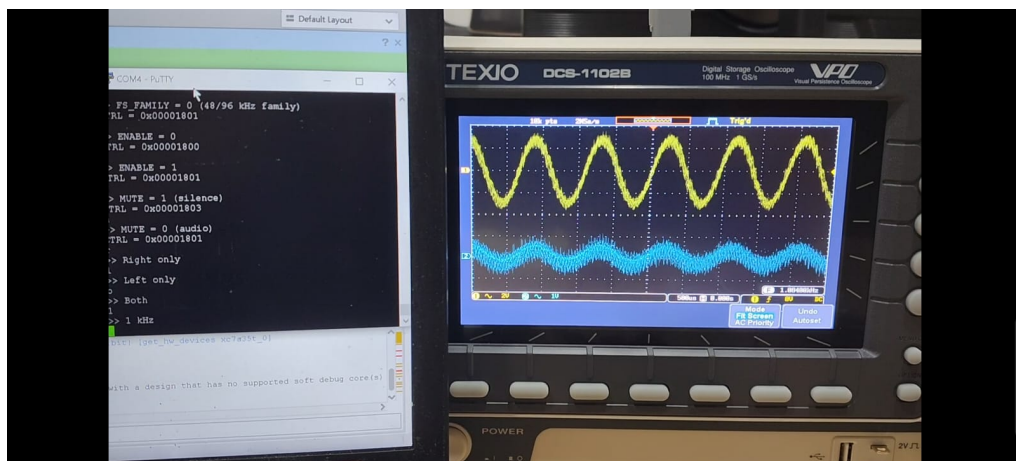
Gambar L.4. Interface view pemetaan pin dan jalur debug.

L.5 Hasil pengujian i2s transmitter - capture osiloskop

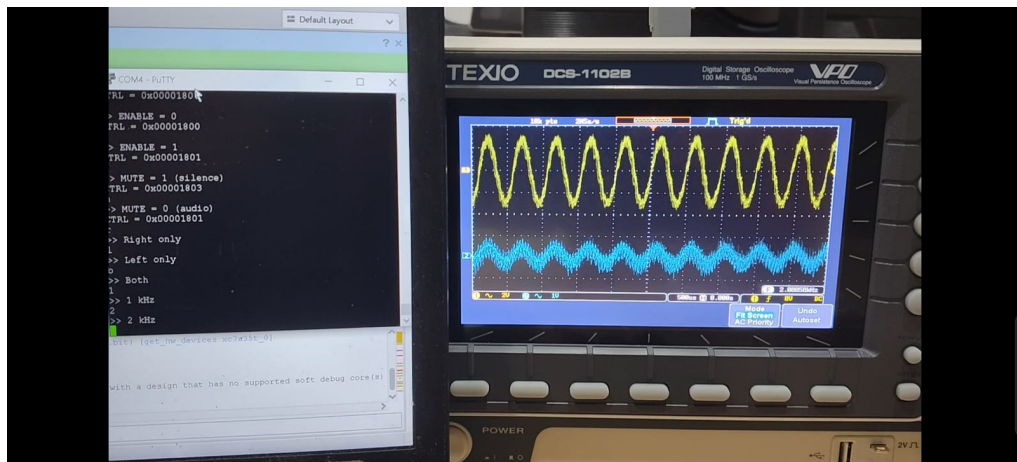
Bagian ini menampilkan hasil pengujian I2S Transmitter melalui capture osiloskop eksternal. Seluruh observasi dilakukan dari sinyal nyata pada probe osiloskop, bukan melalui ILA internal. Gambar berikut memperlihatkan BCLK, LRCLK/WS, dan DATA pada beberapa kondisi uji.

L.5.1 Test Frekuensi Audio

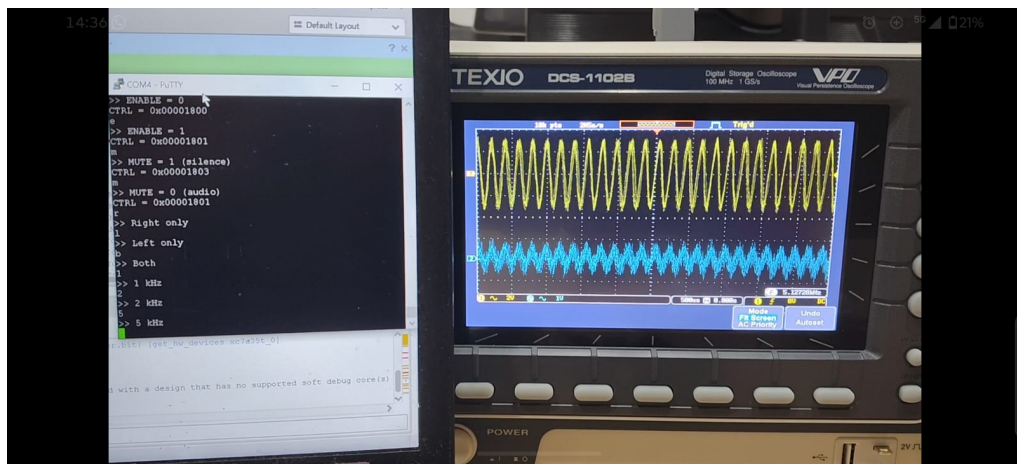
Pengujian frekuensi sinus untuk verifikasi laju sampling dan integritas I2S:



Gambar L.5. Capture osiloskop: audio tone 1 kHz.



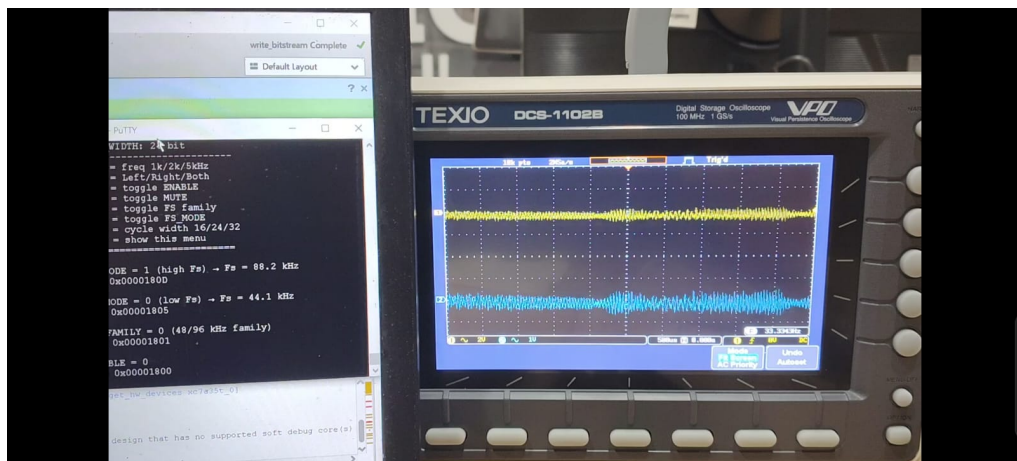
Gambar L.6. Capture osiloskop: audio tone 2 kHz.



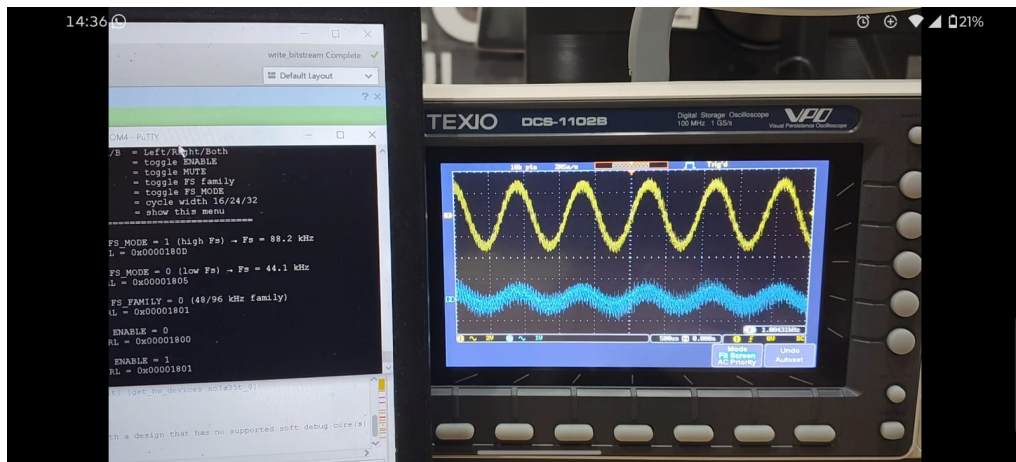
Gambar L.7. Capture osiloskop: audio tone 5 kHz.

L.5.2 Test Kontrol Enable

Pengujian sinyal enable untuk aktivasi dan deaktivasi I2S transmitter:



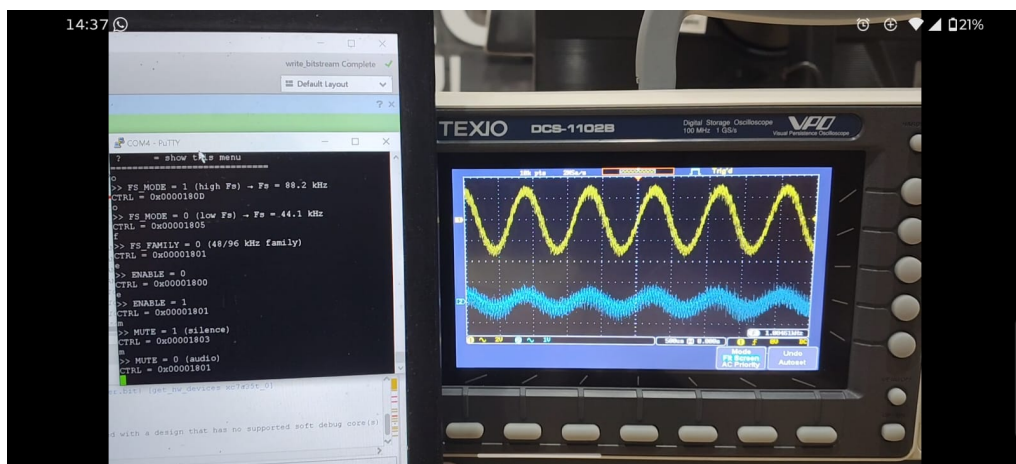
Gambar L.8. Capture osiloskop: enable=0.



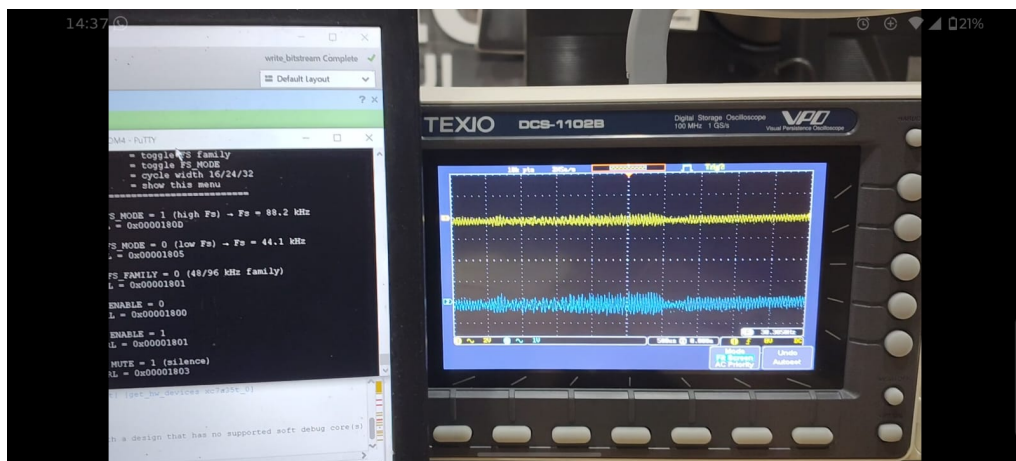
Gambar L.9. Capture osiloskop: enable=1.

L.5.3 Test Kontrol Mute

Pengujian fungsi mute untuk menghentikan keluaran audio sambil mempertahankan sinyal I2S:



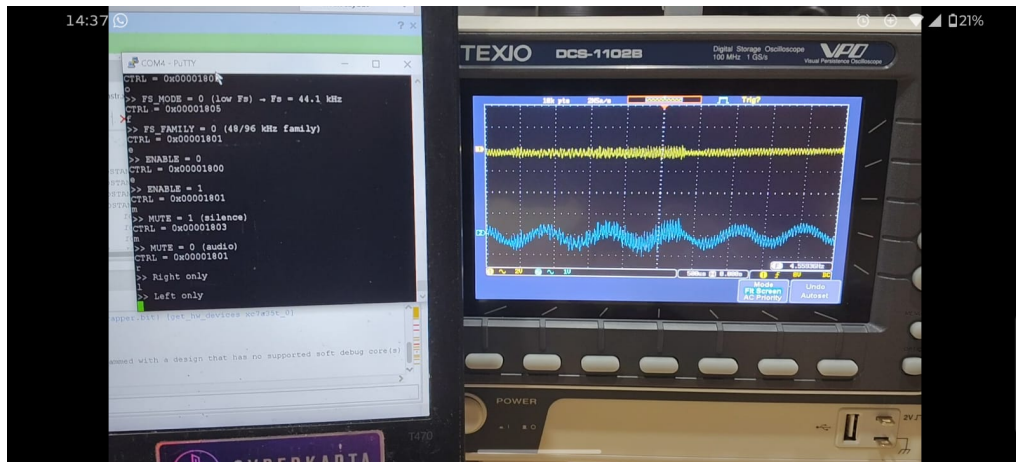
Gambar L.10. Capture osiloskop: mute=0.



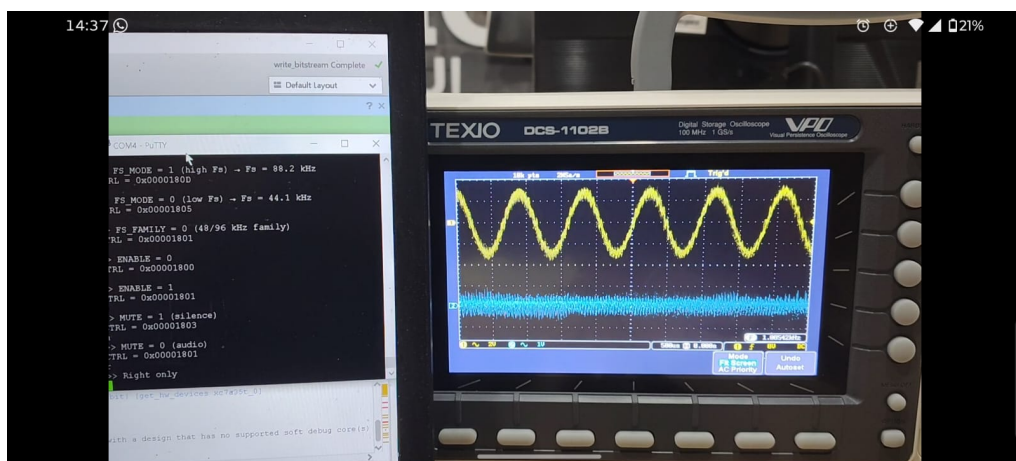
Gambar L.11. Capture osiloskop: mute=1.

L.5.4 Test Routing Stereo

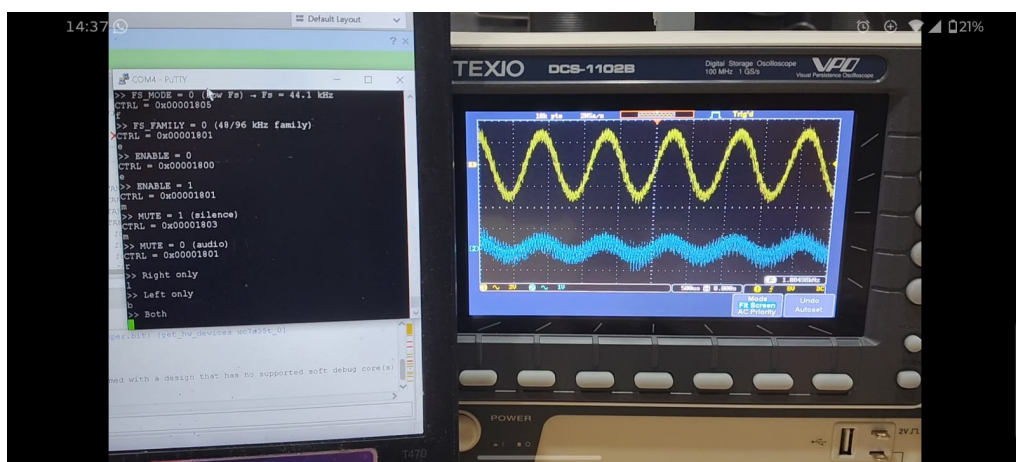
Pengujian routing kanal stereo untuk verifikasi pemisahan left dan right channel:



Gambar L.12. Hasil capture osiloskop: Left channel only - hanya kanal left dioutput



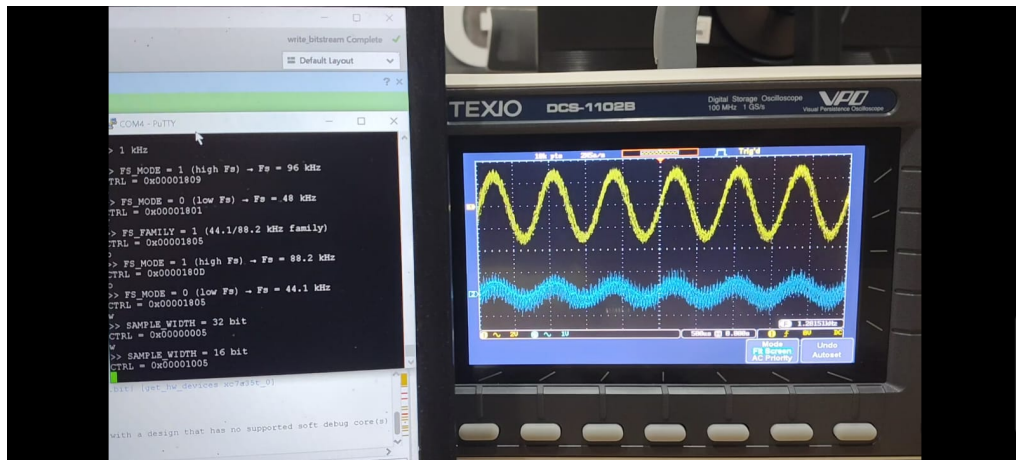
Gambar L.13. Hasil capture osiloskop: Right channel only - hanya kanal right dioutput



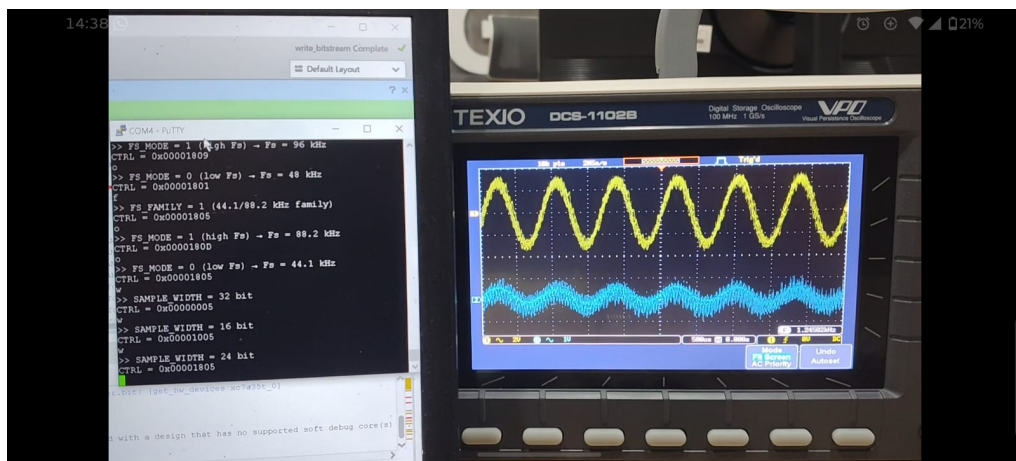
Gambar L.14. Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan

L.5.5 Test Lebar Sample (Sample Width)

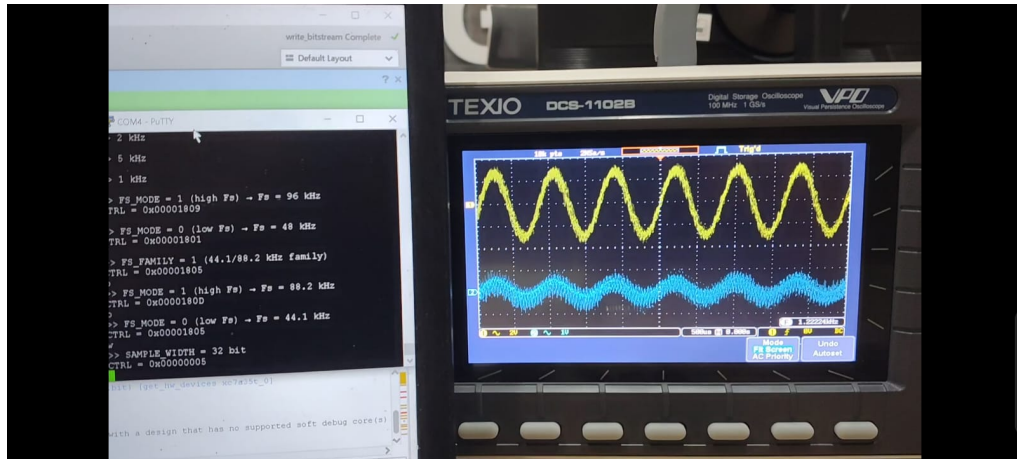
Pengujian dengan berbagai lebar bit untuk verifikasi format audio dan slot frame I2S:



Gambar L.15. Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit



Gambar L.16. Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit



Gambar L.17. Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame

L.6 Design Timing Summary

Laporan timing summary dari hasil implementasi Vivado menunjukkan performa timing keseluruhan desain:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.251 ns	Worst Hold Slack (WHS): 0.039 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7290	Total Number of Endpoints: 7290	Total Number of Endpoints: 2637

All user specified timing constraints are met.

Gambar L.18. Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure

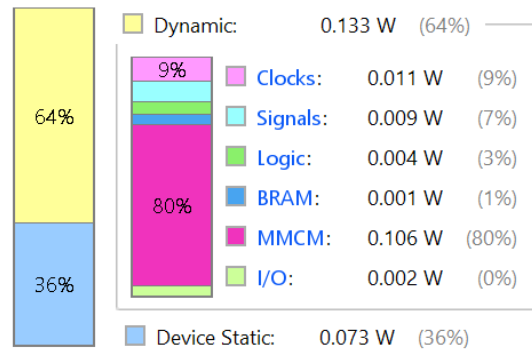
L.7 Power Summary

Laporan analisis power dari post-implementation desain menunjukkan konsumsi daya dinamis dan statis:

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	0.207 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	26.0°C
Thermal Margin:	59.0°C (11.7 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

On-Chip Power



Gambar L.19. Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins

BLOCK DESIGN IP SUMMARIES

This appendix lists the main IP blocks present in the Vivado Block Design `design_1` and summarizes their primary interfaces and key parameters. XML metadata files for each IP are located under `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/`.

design_1_clk_wiz_0_0

Purpose: Clocking Wizard that provides generated clocks and an AXI-lite control interface. Main interfaces: - `s_axi_lite` (AXI4, control) - `s_axi_aclk` (clock) Key parameters: `DATA_WIDTH=32`, `FREQ_HZ=100000000`, `ADDR_WIDTH=32` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_clk_wiz_0_0`

design_1_microblaze_riscv_0_0

Purpose: MicroBlaze RISC-V soft processor for embedded software control. Main interfaces: - Clock/Reset signals (clock domain `/clk_wiz_0_clk_out1`) - `INTERRUPT` / `MON_INTERRUPT` (interrupt inputs) - Multiple AXI/monitor interfaces referenced in `ASSOCIATED_BUSIF` Key parameters: `FREQ_HZ=100000000`, `CLK_DOMAIN=/clk_wiz_0_clk_out1` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_microblaze_riscv_0_0`

design_1_dlmb_v10_0

Purpose: Data Local Memory Bus (DLMB) slave interfaces for processor local memory access. Main interfaces: - `LMB_Sl_0`, `LMB_Sl_1`, ... (LMB mirrored slave ports) Key parameters: `ADDR_WIDTH=32`, `DATA_WIDTH=32`, `READ_WRITE_MODE=READ_WRITE` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_dlmb_v10_0`

design_1_ilmb_v10_0

Purpose: Instruction Local Memory Bus (ILMB) slave interfaces for instruction fetch. Main interfaces: - `LMB_Sl_0`, `LMB_Sl_1`, ... (LMB mirrored slave ports) Key parameters: `ADDR_WIDTH=32`, `DATA_WIDTH=32`, `READ_WRITE_MODE=READ_WRITE` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_ilmb_v10_0`

design_1_dlmb_bram_if_cntlr_0

Purpose: BRAM interface controller connected to DLMB (data-side local memory controller). Main interfaces: - `INTERRUPT` (interrupt output) - `SLMB`, `SLMB1`, ... (LMB slave interfaces) Key parameters: provides memory map refs `SLMB`, data/addr widths 32-bit XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_dlmb_bram_if_cntlr_0`

design_1_ilmb_bram_if_cntlr_0

Purpose: BRAM interface controller connected to ILMB (instruction-side local memory controller). Main interfaces: - LMB slave interfaces (instruction path) Key parameters: standard BRAM/LMB glue with 32-bit widths XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_lmb_bram_0

Purpose: BRAM block exposed as LMB/AXI memory. Main interfaces: - Clock/Reset (ACLK, ARESETN) - AXI_SLAVE_S_AXI (AXI interface to access BRAM) Key parameters: FREQ_HZ=100000000, AXI slave memory map S_1 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_mdm_1_0

Purpose: Microprocessor Debug Module (MDM) providing debug access (AXI-Lite) and control/monitoring. Main interfaces: - S_AXI (AXI4-Lite) Key parameters: DATA_WIDTH=32, ADDR_WIDTH=32 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_rst_clk_wiz_0_100M_0

Purpose: Reset controller synchronizer associated with the clock wizard (generates resets for bus/processor domains). Main interfaces: - clock (slowest_sync_clk) - ext_reset, aux_reset, dbg_reset (external reset inputs) Key parameters: FREQ_HZ=100000000, ASSOCIATED_RESETS=mb_reset:bus_struct_reset:interconnect_aresetn:peripheral_aresetn XML: i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_rst_clk_wiz_0_100M_0

design_1_axi_smc_0

Purpose: AXI SmartConnect / interconnect for routing AXI masters to AXI slaves (protocol conversion, sizing, etc.). Main interfaces: - CLK.aclk (clock) - RST.aresetn (reset) - S00_AXI (AXI slave port) - M00_AXI, M01_AXI (AXI master ports) – routing targets Key parameters: FREQ_HZ=100000000, ASSOCIATED_BUSIF=M00_AXI:M01_AXI:S00_AXI XML: i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_axi_smc_0

design_1_i2s_0_0

Purpose: Custom/third-party I2S transmitter IP wrapped as AXI slave (control via AXI4-Lite). Main interfaces: - S00_AXI (AXI4-Lite control interface, ADDR_WIDTH=4, DATA_WIDTH=32, PROTOCOL=AXI4LITE) Key parameters: WIZ_DATA_WIDTH=32, WIZ_NUM_REG=4, ADDR_WIDTH=4 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

BAB L

KODE SUMBER PERANGKAT LUNAK (VITIS)

Berikut adalah kode sumber utama (helloworld.c) yang digunakan untuk menguji peripheral I2S pada platform Vitis bare-metal.

```
1  /*
2      ****
3      * I2S register feature test – all control bits
4      ****
5      */
6
7  #include <stdint.h>
8  #include "platform.h"
9  #include "xil_printf.h"
10 #include "xil_io.h"
11 #include "xparameters.h"
12 #include "xuartlite_1.h"
13
14 /* Read the RISC-V cycle counter (mcycle CSR) */
15 static inline uint64_t rdcycle64(void)
16 {
17     uint32_t lo, hi, hi2;
18     do {
19         __asm__ volatile ("rdcycleh %0" : "=r"(hi));
20         __asm__ volatile ("rdcycle %0" : "=r"(lo));
21         __asm__ volatile ("rdcycleh %0" : "=r"(hi2));
22     } while (hi != hi2);
23     return ((uint64_t)hi << 32) | lo;
24 }
25
26 /* ----- Register map ----- */
27 #define I2S_BASE      XPAR_I2S_0_BASEADDR
28 #define DATA_LEFT    (I2S_BASE + 0x00U)
29 #define DATA_RIGHT    (I2S_BASE + 0x04U)
30 #define CONTROL        (I2S_BASE + 0x08U)
31
32 /* ----- Control bits ----- */
33 #define CTRL_ENABLE    (1U << 0)
34 #define CTRL_MUTE      (1U << 1)
```

```

33 #define CTRL_FS          (1U << 2)
34 #define CTRL_FS_MODE     (1U << 3)
35 #define CTRL_WIDTH(w)    (((uint32_t)(w) & 0x1FU) << 8U)
36
37 /* ----- Audio ----- */
38 #define SAMPLE_RATE      48000U
39 #define AMPLITUDE        32000
40
41 /* ----- Sine table ----- */
42 static const int16_t SINE[32] = {
43     0, 6393, 12539, 18204, 23170, 27245, 30273, 32137,
44     32767, 32137, 30273, 27245, 23170, 18204, 12539, 6393,
45     0, -6393, -12539, -18204, -23170, -27245, -30273, -32137,
46     -32767, -32137, -30273, -27245, -23170, -18204, -12539, -6393
47 };
48
49 /* ----- State ----- */
50 static uint32_t g_freq   = 1000U;
51 static uint8_t  g_left   = 1U;
52 static uint8_t  g_right  = 1U;
53 static uint8_t  g_enable = 1U;
54 static uint8_t  g_mute   = 0U;
55 static uint8_t  g_fs     = 0U;
56 static uint8_t  g_fs_mode = 0U;
57 static uint8_t  g_width  = 24U;
58
59 static uint32_t current_sample_rate(void)
60 {
61     if (!g_fs && !g_fs_mode) return 48000U;
62     if ( g_fs && !g_fs_mode) return 44100U;
63     if (!g_fs && g_fs_mode) return 96000U;
64     return 88200U;
65 }
66
67 static void apply_control(void)
68 {
69     uint32_t ctrl = 0U;
70     if (g_enable) ctrl |= CTRL_ENABLE;
71     if (g_mute)   ctrl |= CTRL_MUTE;
72     if (g_fs)     ctrl |= CTRL_FS;
73     if (g_fs_mode) ctrl |= CTRL_FS_MODE;

```

```

74 ctrl |= CTRL_WIDTH(g_width);
75 Xil_Out32(CONTROL, ctrl);
76 }
77
78 static void stream_sample(void)
79 {
80     static uint32_t phase = 0U;
81     uint32_t sr = current_sample_rate();
82     uint32_t step = (uint32_t)(((uint64_t)g_freq << 32) / sr);
83
84     int16_t s = (int16_t)(((int32_t)SINE[phase >> 27] * AMPLITUDE) / 32767);
85
86     int32_t s32;
87     if (g_width >= 32U)    s32 = (int32_t)s << 16;
88     else if (g_width > 16U) s32 = (int32_t)s << (g_width - 16U);
89     else if (g_width < 16U) s32 = (int32_t)s >> (16U - g_width);
90     else                  s32 = (int32_t)s;
91
92     uint32_t samp = (uint32_t)s32;
93
94     Xil_Out32(DATA_LEFT, g_left ? samp : 0U);
95     Xil_Out32(DATA_RIGHT, g_right ? samp : 0U);
96     phase += step;
97 }
98
99 int main(void)
100 {
101     init_platform();
102     apply_control();
103
104     uint64_t t_deadline = rdcycle64();
105
106     while (1) {
107         uint32_t sample_rate = current_sample_rate();
108         uint64_t period_cycles = (uint64_t)XPAR_CPU_CORE_CLOCK_FREQ_HZ / sample_rate
109         ;
110
111         stream_sample();
112
113         if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS)) {
114             uint8_t ch = XUartLite_RecvByte(STDIN_BASEADDRESS);

```

```
114     // Handle keypress logic...
115 }
116
117 t_deadline += period_cycles;
118 while (rdcycle64() < t_deadline) {}
119 }
120
121 cleanup_platform();
122 return 0;
123 }
```

Listing L.1. Kode sumber helloworld.c untuk pengujian I2S

BAB L

KODE SUMBER PERANGKAT KERAS (VERILOG)

L.1 Top-level Wrapper (i2s.v)

```

1  `timescale 1 ns / 1 ps
2
3  module i2s #
4  (
5      parameter integer C_S00_AXI_DATA_WIDTH = 32,
6      parameter integer C_S00_AXI_ADDR_WIDTH = 4
7  )
8  (
9      input  wire audio_48_clk,
10     input  wire audio_44_clk,
11     output wire i2s_mclk,
12     output wire i2s_bclk,
13     output wire i2s_ws,
14     output wire i2s_data,
15     input  wire          s00_axi_aclk,
16     input  wire          s00_axi_aresetn,
17     input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_awaddr,
18     input  wire [2:0]          s00_axi_awprot,
19     input  wire          s00_axi_awvalid,
20     output wire          s00_axi_awready,
21     input  wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_wdata,
22     input  wire [(C_S00_AXI_DATA_WIDTH/8)-1:0] s00_axi_wstrb,
23     input  wire          s00_axi_wvalid,
24     output wire          s00_axi_wready,
25     output wire [1:0]          s00_axi_bresp,
26     output wire          s00_axi_bvalid,
27     input  wire          s00_axi_bready,
28     input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_araddr,
29     input  wire [2:0]          s00_axi_arprot,
30     input  wire          s00_axi_arvalid,
31     output wire          s00_axi_arready,
32     output wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_rdata,
33     output wire [1:0]          s00_axi_rresp,
34     output wire          s00_axi_rvalid,
35     input  wire          s00_axi_rready
36 );

```

```

37
38 i2s_slave_lite_v1_0_S00_AXI #(
39     .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
40     .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
41 ) i2s_slave_lite_v1_0_S00_AXI_inst (
42     .audio_48_clk    (audio_48_clk),
43     .audio_44_clk    (audio_44_clk),
44     .i2s_mclk        (i2s_mclk),
45     .i2s_bclk        (i2s_bclk),
46     .i2s_ws          (i2s_ws),
47     .i2s_data        (i2s_data),
48     .S_AXI_ACLK      (s00_axi_aclk),
49     .S_AXI_ARESETN   (s00_axi_aresetn),
50     .S_AXI_AWADDR    (s00_axi_awaddr),
51     .S_AXI_AWPROT    (s00_axi_awprot),
52     .S_AXI_AWVALID   (s00_axi_awvalid),
53     .S_AXI_AWREADY   (s00_axi_awready),
54     .S_AXI_WDATA     (s00_axi_wdata),
55     .S_AXI_WSTRB     (s00_axi_wstrb),
56     .S_AXI_WVALID    (s00_axi_wvalid),
57     .S_AXI_WREADY    (s00_axi_wready),
58     .S_AXI_BRESP     (s00_axi_bresp),
59     .S_AXI_BVALID    (s00_axi_bvalid),
60     .S_AXI_BREADY    (s00_axi_bready),
61     .S_AXI_ARADDR    (s00_axi_araddr),
62     .S_AXI_ARPROT    (s00_axi_arprot),
63     .S_AXI_ARVALID   (s00_axi_arvalid),
64     .S_AXI_ARREADY   (s00_axi_arready),
65     .S_AXI_RDATA     (s00_axi_rdata),
66     .S_AXI_RRESP     (s00_axi_rresp),
67     .S_AXI_RVALID    (s00_axi_rvalid),
68     .S_AXI_RREADY    (s00_axi_rready)
69 );
70
71 endmodule

```

Listing L.1. Kode sumber i2s.v

L.2 Modul AXI4-Lite Slave dan Serializer

(i2s_slave_lite_v1_0_S00_AXI.v)

```

1 // Glitch-free clock multiplexer

```

```

2 module clk_mux (
3     input wire clk0, clk1, sel,
4     output wire clk_o
5 );
6     reg q0_d1, q0_d2, q1_d1, q1_d2;
7     always @(negedge clk0) begin
8         q0_d1 <= !sel & !q1_d2;
9         q0_d2 <= q0_d1;
10    end
11    always @(negedge clk1) begin
12        q1_d1 <= sel & !q0_d2;
13        q1_d2 <= q1_d1;
14    end
15    assign clk_o = (q0_d2 & clk0) | (q1_d2 & clk1);
16 endmodule
17
18 // Clock Selection Logic
19 wire reg_fs_family = slv_reg2[2];
20 wire internal_mclk;
21
22 clk_mux mclk_mux_inst (
23     .clk0 (audio_48_clk),
24     .clk1 (audio_44_clk),
25     .sel (reg_fs_family),
26     .clk_o (internal_mclk)
27 );
28
29 // CDC Sync
30 cdc_sync #(.WIDTH(32)) cdc_ctrl_inst (
31     .dest_clk (internal_mclk),
32     .src_in (slv_reg2),
33     .dest_out (ctrl_cdc_w)
34 );
35
36 // Serializer function
37 function i2s_next_serial_bit;
38     input [31:0] left_samp;
39     input [31:0] right_samp;
40     input [5:0] bc;
41     input [5:0] width;
42     reg in_left;

```

```

43 reg [4:0] pos;
44 reg [5:0] bit_idx;
45 begin
46     in_left = (bc < 6'd32);
47     pos = bc[4:0];
48     if (width == 6'd0)
49         i2s_next_serial_bit = 1'b0;
50     else if (pos == 5'd0)
51         i2s_next_serial_bit = 1'b0;
52     else if (pos <= width) begin
53         bit_idx = width - {1'b0, pos};
54         i2s_next_serial_bit = in_left ? left_samp[bit_idx[4:0]]
55                                 : right_samp[bit_idx[4:0]];
56     end else
57         i2s_next_serial_bit = 1'b0;
58 end
59 endfunction
60
61 // I2S output generator
62 always @(posedge internal_mclk or posedge audio_rst) begin
63     if (audio_rst) begin
64         bit_count_q <= 6'd0;
65         data_q <= 1'b0;
66         ws_q <= 1'b0;
67         bclk_q <= 1'b0;
68     end else if (bclk_en_w) begin
69         if (!enable_sync) begin
70             bit_count_q <= 6'd0;
71             data_q <= 1'b0;
72             ws_q <= 1'b0;
73             bclk_q <= 1'b0;
74         end else begin
75             if (bclk_q) begin
76                 bclk_q <= 1'b0;
77                 data_q <= i2s_next_serial_bit(
78                     sample_for_i2s_left,
79                     sample_for_i2s_right,
80                     bit_count_q,
81                     sample_width_sync);
82                 ws_q <= bit_count_q[5];
83                 bit_count_q <= bit_count_q + 6'd1;

```

```
84         end else begin
85             bclk_q <= 1'b1;
86         end
87     end
88 end
89 end
```

Listing L.2. Potongan kode i2s_slave_lite_v1_0_S00_AXI.v